

28.04.2026

Алгоритмы работы со внешней памятью.

Филиппов Михаил Витальевич

m.filippov@g.nsu.ru

89232283872

Императивное программирование, 2025-2026

N * Новосибирский
государственный
университет
***НАСТОЯЩАЯ НАУКА**

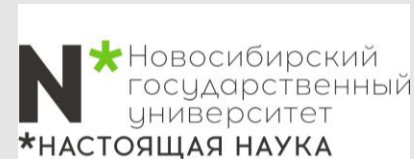


Давайте познакомимся



Филиппов Михаил Витальевич

- Окончил магистратуру ФФ НГУ
- Окончил аспирантуру ИТ СО РАН
- Являюсь м.н.с. ИТ СО РАН
- 8+ лет опыт в программировании C/C++
- к.ф.-м.н.



План презентации

**Алгоритмы
работы со
внешней
памятью**

90 минут

Введение

Если данных много – алгоритмы работающие с малым количеством данных могут оказаться не применимы для большого количества данных.

Internal Memory – внутренняя память (RAM)

External Memory – внешняя память (HDD, Flash)

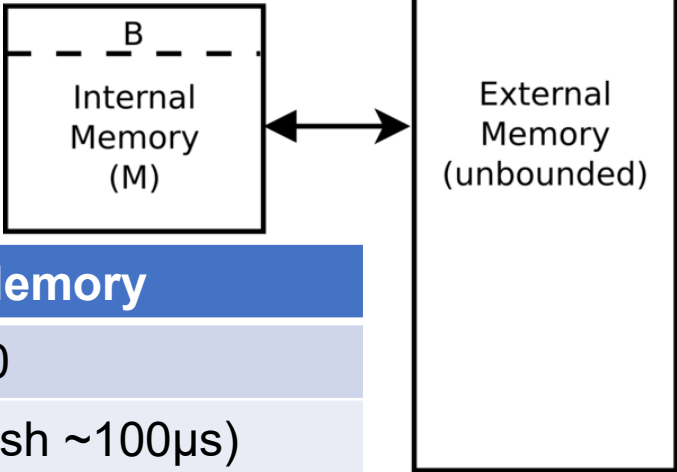
Очевидно для некоторых задач необходимая память >> Internal Memory.

Сервер:

Space – пространство

Latency – задержка перед началом чтения

Throughput – пропускная способность



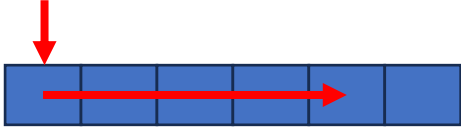
	Internal Memory	External Memory
Space	~ 1Tb	~ 10Tb * 10
Latency	~ 100 ns	~ 10ms (flash ~100μs)
Throughput	~ 10Gb/s	~ 100Mb/s (flash ~1Gb/s)



Модель для скорости алгоритмов

Когда не было внешней памяти – RAM модель.

RF – это регистр



$$\text{time}(x) = \text{latency} + x/\text{throughput}$$

Когда вводится диск, считаются только операции ввода – вывода

Пусть M – размер оперативной памяти, N – размер внешней памяти (можно иначе размер входа), B – размер блока.

За единицу времени вы можете считать и записать блок B .

Хотим модель $\text{latency} \sim x/\text{throughput}$ для упрощения модели.

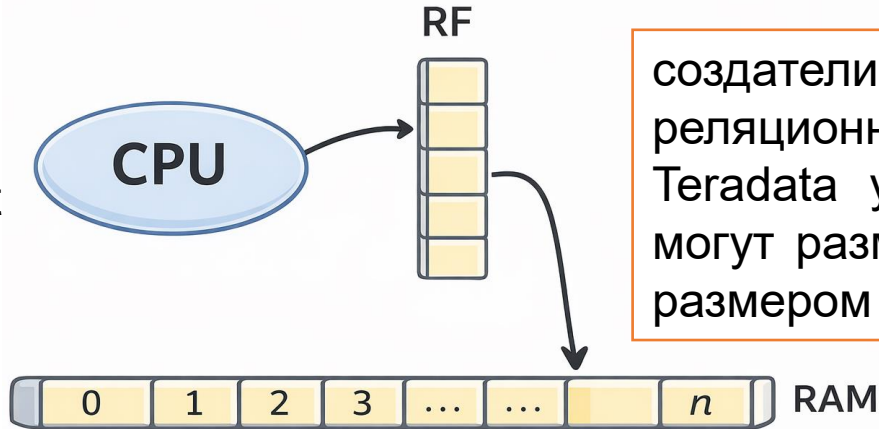
$x > B$; $B(\text{HDD}) \sim 1 \text{ Mb}$; $B(\text{Flash}) \sim 100 \text{ Kb}$

У нас значительно лучше.

Latency увеличить – ускорить скорость вращения

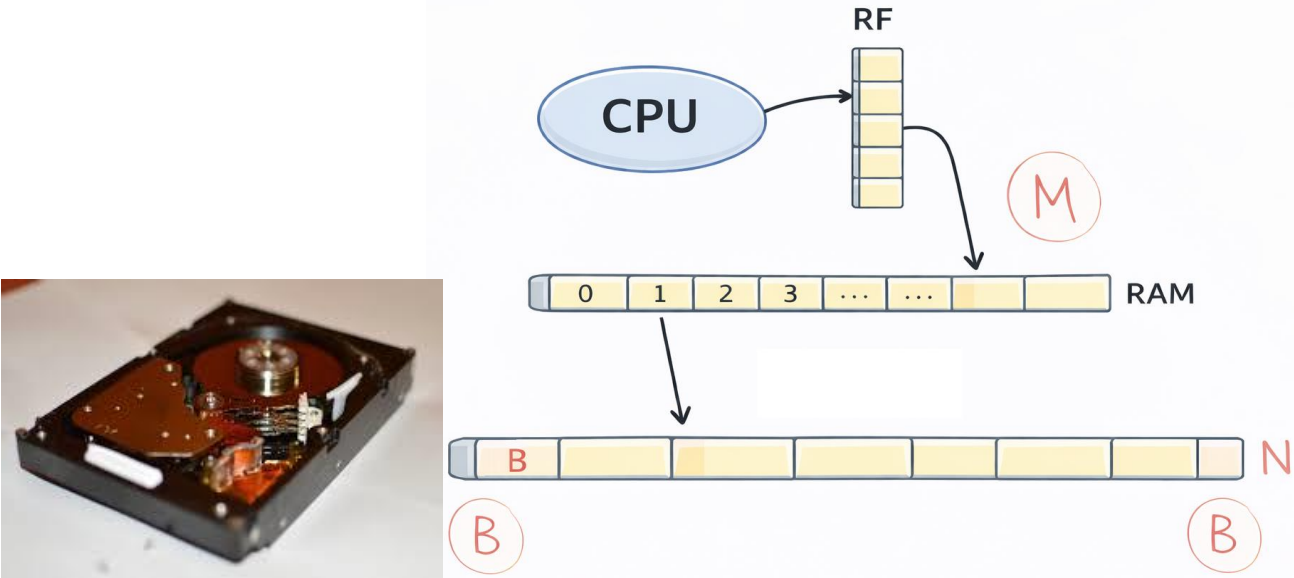
Throughput увеличить – увеличить количество блинов или дисков

RAM Model



создатели системы управления реляционными базами данных Teradata утверждают, что они могут размещать базы данных размером до 50 петабайт (Пб).

EM Model



Задачи – примеры.

1. Дан массив $a_1 \dots a_N$. Найти сумму чисел. Очевидно читаете и считаете поблочно. Очевидно сложность равна количеству операций ввода вывода. $IOs = \lceil N/B \rceil = \text{Scan}(N)$.

read buffer



B

write buffer



B

Вспоминаем что у нас есть буферизация (она даже бывает в жестком диске).

2. ListRankig – походить по списку (нетривиально) $IOs = O(N) \rightarrow O(\text{Sort}(N))$.
3. Задача сортировки – знаем merge, quick, hear и т. д. Что же теперь с ними делать?

При построении hear мы читаем из разных сегментов памяти, а не непрерывно.

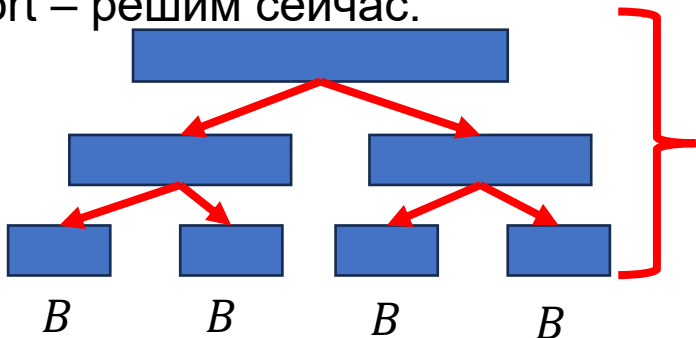
quicksort – возможен, но с дополнительными входными данными. Он будет проигрывать оптимальному mergesort.

mergesort – решим сейчас.

N/B

N/B

N/B



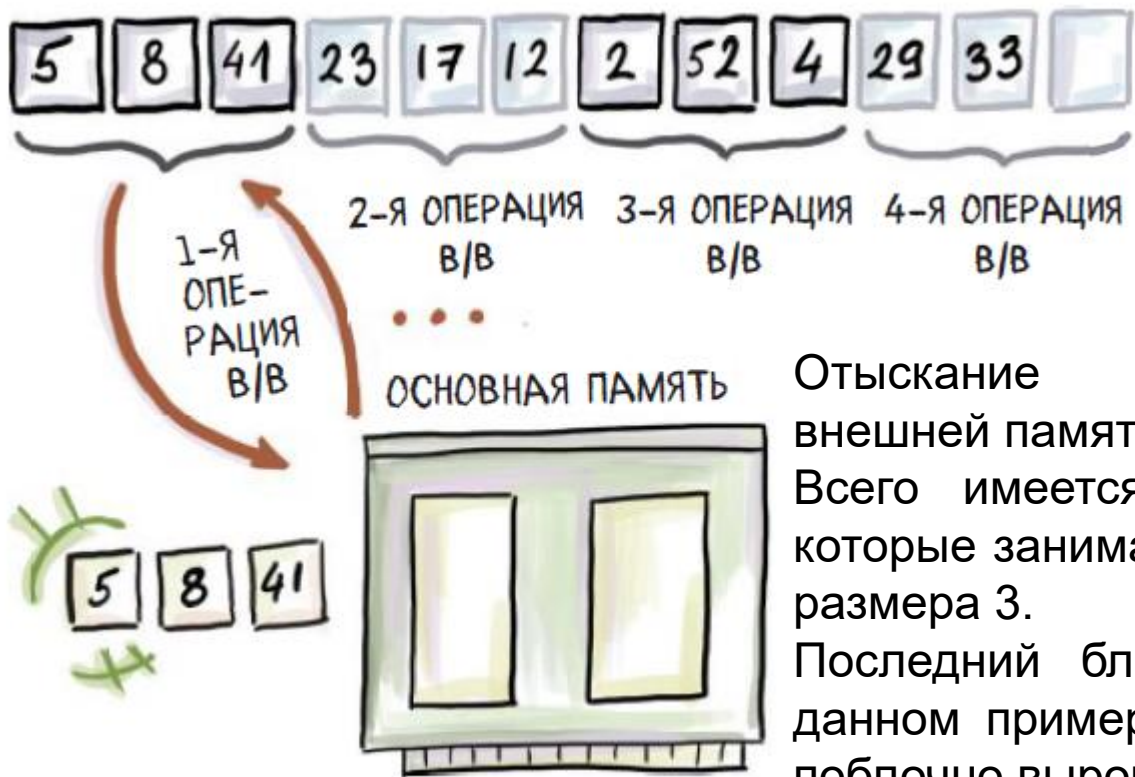
$$\frac{N}{B} \log_2 \frac{N}{B}$$

Стандартно

Можно быстрее, как?



Отыскание минимума

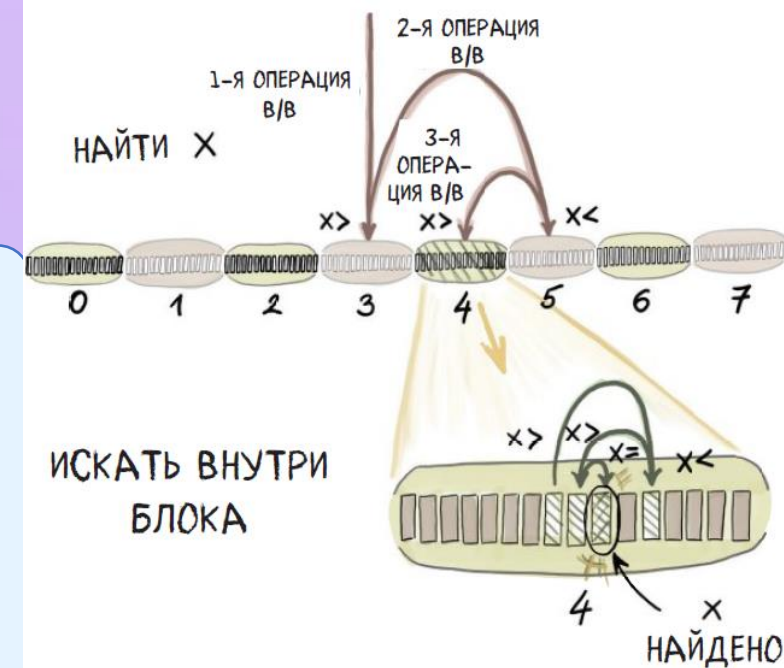


$$IOs = \lceil N/B \rceil = \text{Scan}(N).$$



Двоичный поиск

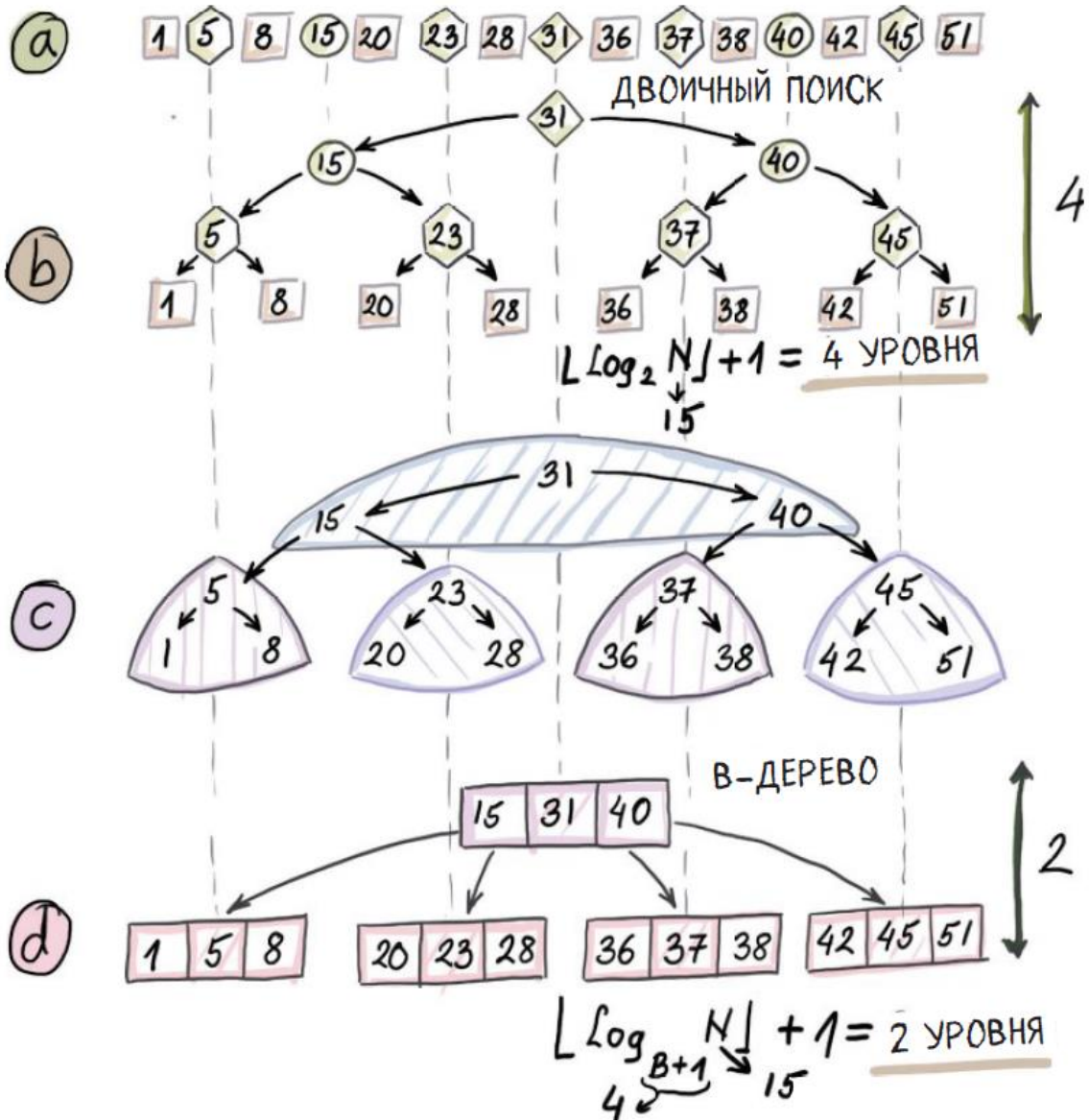
```
#define BLOCK_SIZE 1024
int binarySearchExtMem(const char *filename, long file_start, long
left, long right, int x)
{
    while (left + BLOCK_SIZE <= right)
    {
        long mid = left + (right - left) / 2;
        int BLOCK[BLOCK_SIZE];
        read_block(filename, file_start + mid, BLOCK, BLOCK_SIZE);
        if (BLOCK[BLOCK_SIZE - 1] < x)
            left = mid + 1;
        else if (BLOCK[0] > x)
            right = mid - 1;
        else // Элемент может быть внутри этого блока
            return binarySearch(BLOCK, 0, BLOCK_SIZE - 1, x);
    }
    int BLOCK1[BLOCK_SIZE];
    read_block(filename, file_start + left, BLOCK1, BLOCK_SIZE);
    return binarySearch(BLOCK1, 0, BLOCK_SIZE - 1, x);
}
```



Двоичный поиск во внешней памяти. Алгоритм обращается к отдельному блоку по каждой имеющейся опорной точке, за исключением нескольких последних опорных точек, все из которых находятся в одном блоке.

$$IOs = \log_2 N / B.$$

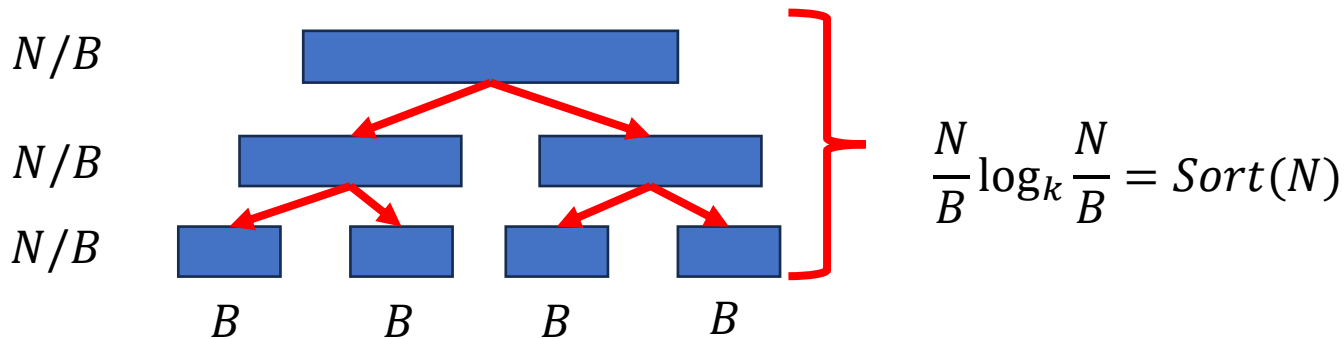
Оптимальный поиск



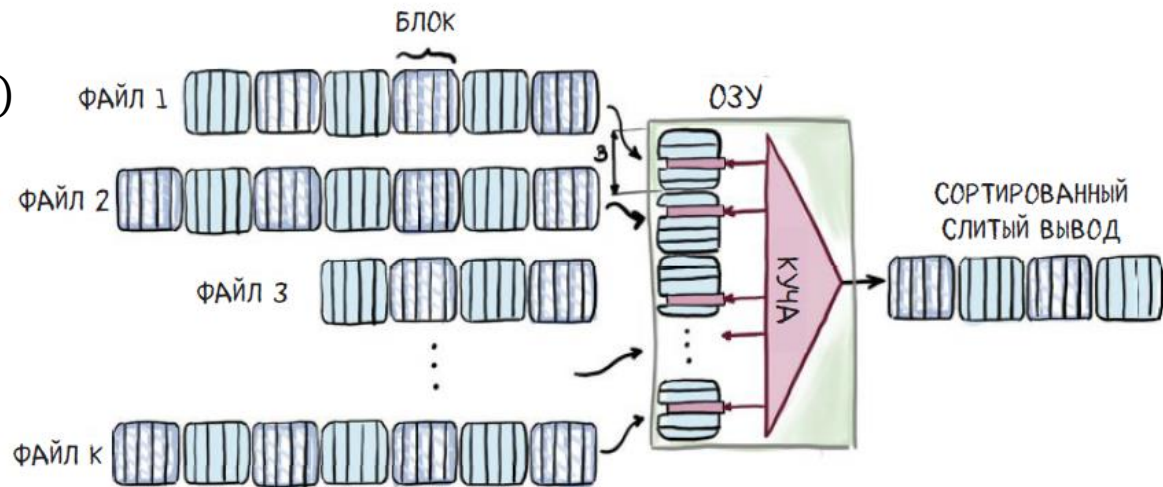
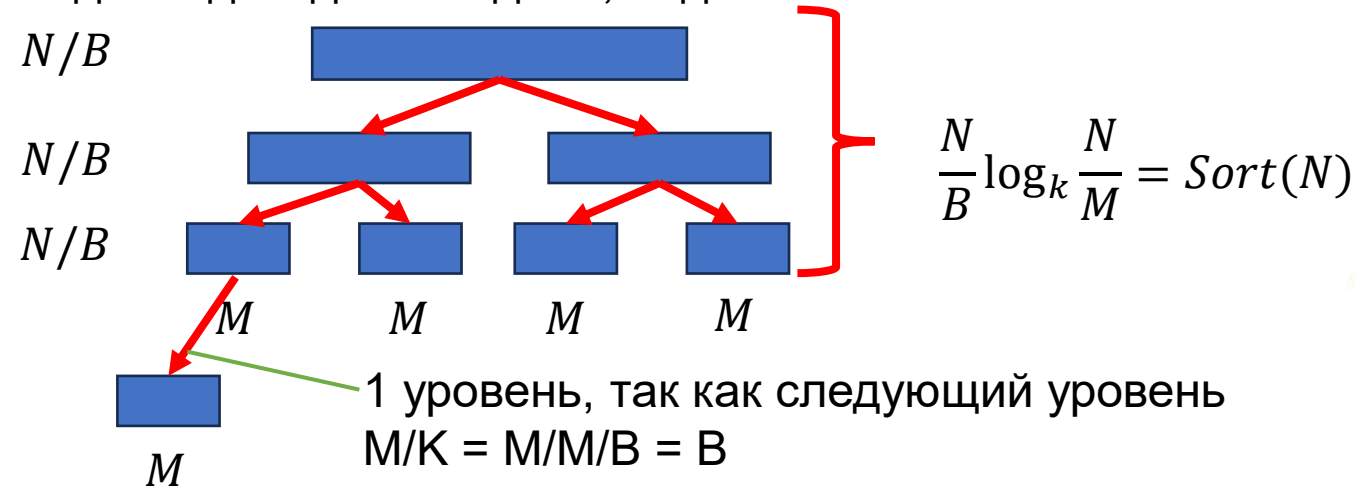
Как преобразовать сортированный массив в структуру, обеспечивающую оптимальный поиск во внешней памяти (В-дерево). Оптимальная поисковая структура данных в оперативной памяти выглядит примерно как d, где у каждого узла есть одна опорная точка, основанная на выбранном узлом пути движения вниз по дереву. Оптимальная поисковая структура данных во внешней памяти имеет большие узлы (размером с блок), которые содержат много элементов, потому что данные извлекаются в память поблочно.

Быстрый MergeSort

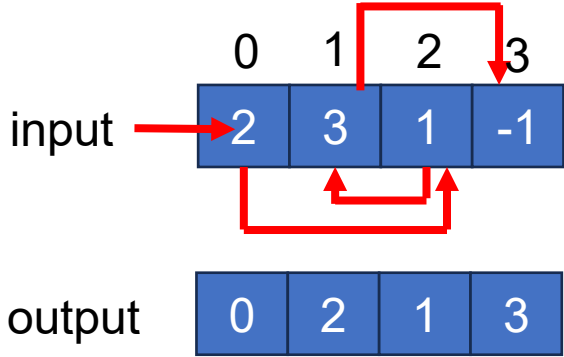
1. Идея – сливать не по 2 блока а сразу по $k \leq M/B$, где M – доступный размер оперативной памяти



2. Идея – доходить не до M , а до B



ListRanking



Неэффективное решение $IOs = O(N)$

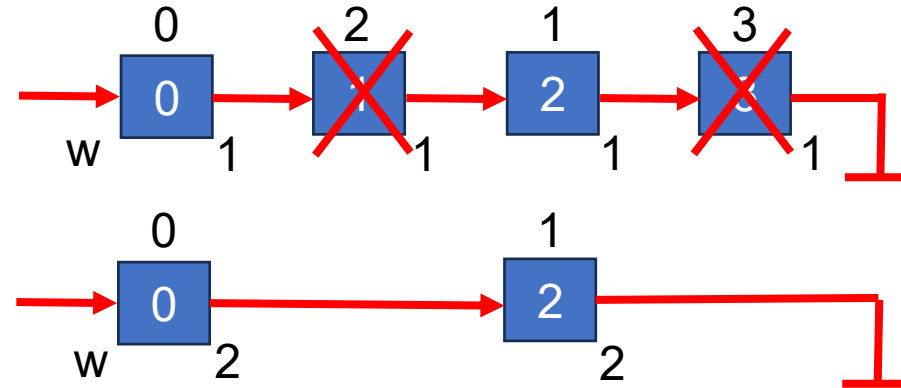
Хотим $O(\text{Sort}(N)) = O\left(\frac{N}{B} \log_k \frac{N}{M}\right)$

Идея 1:

«разделяй и властвуй» - непонятна рекурсия?

Идея 2:

1. Представляем список логически
2. Прореживаем список и перестроить связи естественным образом
3. Подать на вход самому алгоритму и посчитать ответ, узнав ранги оставшихся.



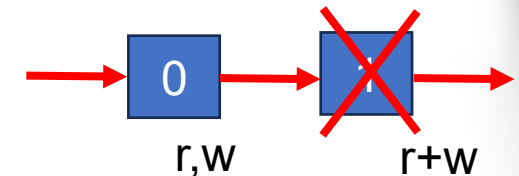
Хочу при этом: Узнать ранги и удаленных и оставшихся реальные.

Переформулируем задачу: любой ранг на нижнем рисунке, должен быть равен рангу на верхнем рисунке. Нужно добавить веса (удаленный элемент должен отдать вес левому элементу).

Удаленные элементы можно восстановить, зная предыдущие.

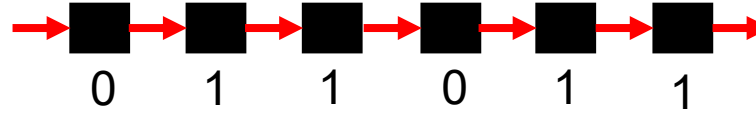
Сколько удалять? Мало – скорость алгоритма низкая, 2 подряд – сложно восстановить удаленные + память непонятна..

$N/2 \rightarrow$ много из-за поиска в памяти.. Есть способы $N/4-N/3$



Хотим итог: $c(N) = O(\text{sort}(N) + \text{sort}(\alpha N) + \text{sort}(\alpha^2 N) + \dots) + c(\alpha N) = O(\text{sort}(N)) + c(\alpha N), 0 < \alpha < 1$

ListRanking



Рандомизированный способ:

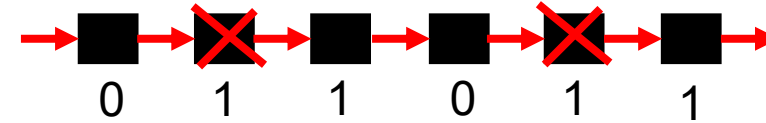
1 этап: Рандомизировано – пишем числа от 0 до 1 ко всем элементам списка

Удаляем например все 1.

2 этап: постпроцессинг – элемент удаляется, если перед 1 есть 0.

Очевидно, что **количество удаленных элементов** $\frac{1}{4}$

Линейность матожидания не требует независимости!

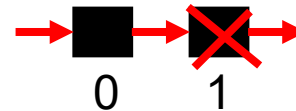


Как эффективно посчитать множество удаляемых элементов?

1. Мы будем работать с датасетами (2D-таблицами).

Будем считать лист – массив пар $\langle i, \text{next}(i) \rangle$.

Хотим сделать такой датасет $\langle i \mid i \in \text{Deleted} \rangle$ **Как?**



(очевидно
чтение
поблочно)

1. Сделать случайный битовый датасет $\langle i \mid b(i) \rangle$

2. Построить датасет $\langle i, b(i), \text{next}(i), \text{next}(b(i)) \rangle$

3. С помощью линейного сканирования $b(i), \text{next}(b(i))$ получить $\langle i \mid i \in \text{Deleted} \rangle$

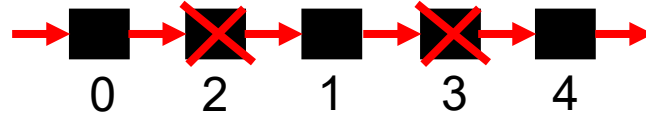
Как получить хотя бы $\langle i, b(i), \text{next}(i) \rangle$?

Есть 2 датасета $\langle i, \text{next}(i) \rangle$ и $\langle i \mid b(i) \rangle \rightarrow$ сортируем по общему ключу и сделать их JOIN по i .

Как получить $\langle i, b(i), \text{next}(i), \text{next}(b(i)) \rangle$?

Есть 2 датасета $\langle i, b(i), \text{next}(i) \rangle$ и $\langle j \mid b(j) \rangle \rightarrow$ сортируем первый по $\text{next}(i)$ и второй по j и сделать их JOIN.

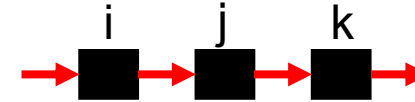
ListRanking



Как получить после удаления из $\langle i, next(i) \rangle \rightarrow \langle j, next'(j) \rangle$?

Чтобы понять все про списки, нужно окно размера 3.

$\langle i, next(i), next^2(i), d(next(i)), d(i) \rangle$, d – флаг удаляется ли.



Алгоритм удаления: $if\ d(i) = F \rightarrow \begin{cases} if\ d(j) = T \rightarrow emit\ \langle i, k \rangle \\ if\ d(j) = T \rightarrow emit\ \langle i, j \rangle \end{cases}$. *emit* – проверить на выходе

Очевидно $\langle i, next(i), next^2(j), d(next(i)), d(i) \rangle$, находится JOIN

Детерминированный алгоритм за $N/3$

Очевидно есть некоторые алгоритмы во внутренней памяти, будет работать для внешней. Свойства:

1. Последовательный доступ к памяти

2. (Неочевидное) Есть граф вычисления.

Ребра потока информации, передающиеся между значениями (вершинами). В каждой вершине

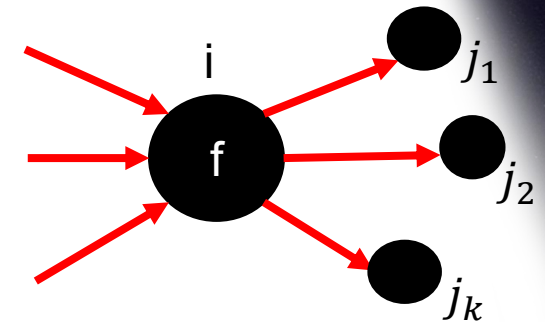
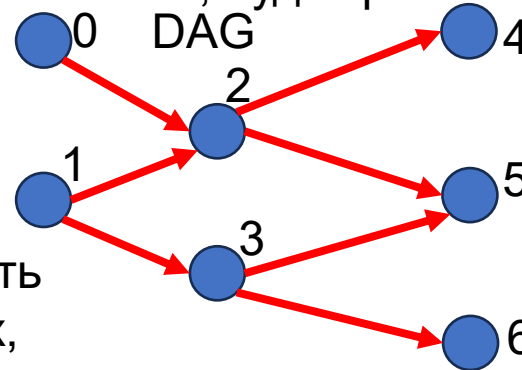
считается некоторая функция f – мы можем получить информацию о том, какие значения в тех вершинах,

которые связаны с текущей вершиной ребрами. Например, все значения, посчитанные предками. Функция производит вычисление и далее это значение распространяется по графу и становится доступно всем вершинам, соединённым ребрами.

Пусть DAG, где вершины пронумерованы топологически. $f(i) \rightarrow (j_1, \dots, j_k)$ знает про $j_1, \dots, j_k$. $\forall j > i$.

Тогда это можно посчитать во внешней памяти за $O(\text{sort}(V + E))!!!$

Определение 29.1. Time forward processing - эффективное вычисление функции f .



ListRanking

Почему модель 2 интересна нам?

Перейдем от связного списка к неориентированному графу:

Никакая пара удаляемых вершин не является соседями в связном списке → в графе никакая из удаляемых вершин не соединена ребром.

Определение 29.2. Независимое множество (stable set) - подмножество графов, в котором в графе никакая из удаляемых вершин не соединена ребром.

$$\max |stable\ set| \geq \frac{|V|}{\max deg + 1}$$

Вспомним задачу о раскраске или нез.множестве из 27 лекции. $k\ colors \leftrightarrow \max |stable\ set| \geq \frac{V}{k}$

Оказывается $k \leq maxdeg + 1$ – очевидно что, если 2 ребра из каждого выходит, то 3 цвет сама вершина. Наш граф можно покрасить и в 2 раза, но сложности с алгоритмом покраски (если начали параллельно красить с нескольких мест).

Для $\forall$ графа можем посмотреть все цвета in ребер в f и покрасить в первый попавшийся из оставшихся.

Какая структура данных поможет нам сделать алгоритм во внешнюю память?

1 вариант multimap: Ключ – номер вершины, а значения – входящие ребра.

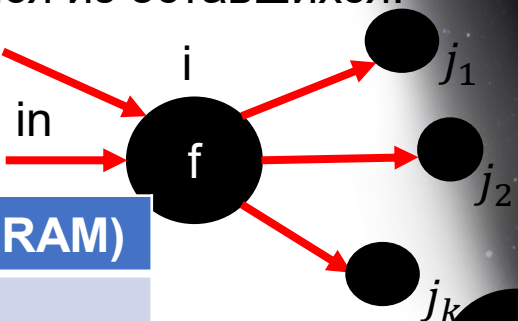
Наше дерево такое не дотянет до $O(\text{sort}(V + E))$ так как количество операций в

дереве будет $V + E$ (get + insert).

Скорость будет $O(B \text{ sort}(V + E))$

Нужна новая СД!!!

	Tree (RAM)	HT (RAM)	Tree (EM)	HT (RAM)
Insert(k,v)	Log N	1	Log N	1
Get(k)	Log N	1	Log N	1



ListRanking

Делаем новую структуру данных

Для вставки – **буферизация** ускорит нашу жизнь.

Для поиска так нельзя, но попробуем просто сделать быстрые Get().

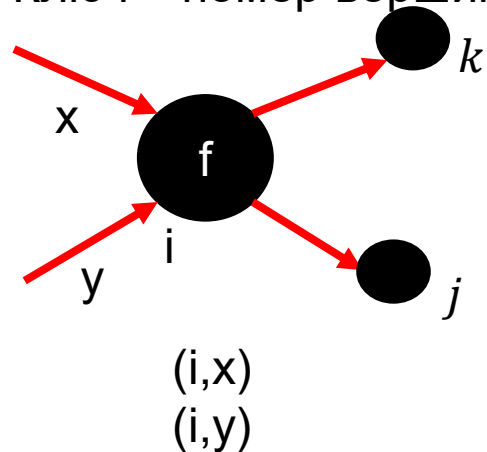
- Можно воспользоваться кэшированием, но это не дает гарантии.
- **Итог:** скорость GET никогда не будет < 1 .

Можно сделать интерфейс асинхронным `async-get(k)`. Мы выдаем обещание посчитать `get` когда-нибудь потом, когда нам будет выгодно. Толк возможен при корреляции запросов.. Сходить в некоторый блок и сразу выдай всем ответ.

Для Time forward processing нужна другая структура данных. Нужна структура данных, которая сама бы выбирала ключ, по которому считать. Пользователь не решает!

Ответ: мульти-куча

Ключ – номер вершины, значение – значение, приезжающее в вершину по ребру.



Что нужно сделать с мультикучей, чтобы обработать вершину i ?

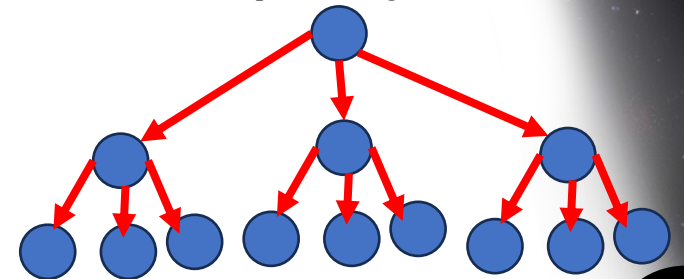
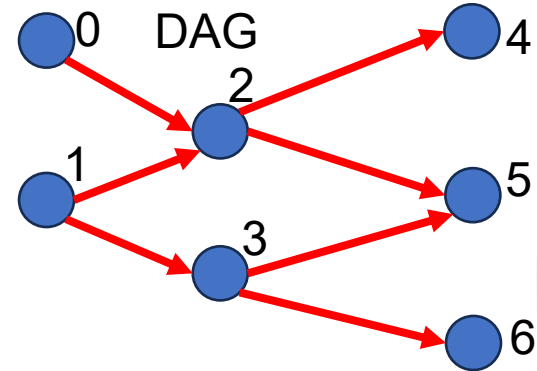
Нужно достать все пары с минимальным ключом.

Итого: multiheap

Будет иметь ассимптотику: $O\left(\frac{1}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right)$ per item

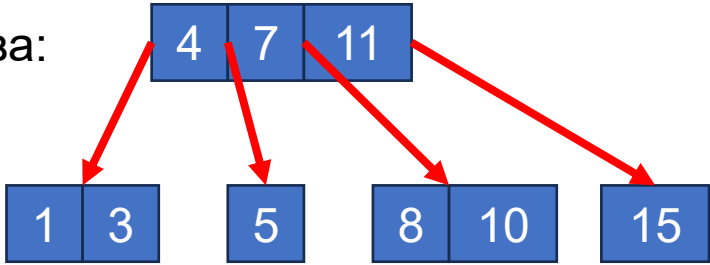
Делаем решение через multiheap:

Куча может быть построена на основе (α, b) -tree, где ключи – листья.



(α, b)-tree

Пример (2,4)-дерева:



В-дерево — это (a,b)-дерево, у которого $a = \text{ceil}(b/2)$.

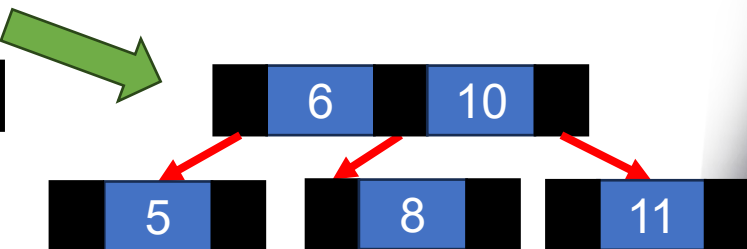
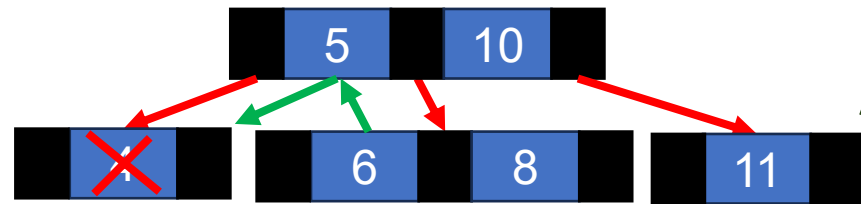
(a,b)-дерево — это сбалансированное дерево поиска, в котором:

- $2 \leq a \leq (b+1)/2$
- Каждый внутренний узел, за исключением корневого, имеет как минимум **a** дочерних узлов и как максимум **b** дочерних узлов.
- Корень имеет не более **b** дочерних элементов.

Вставка числа 10 в (2,4) - дерево

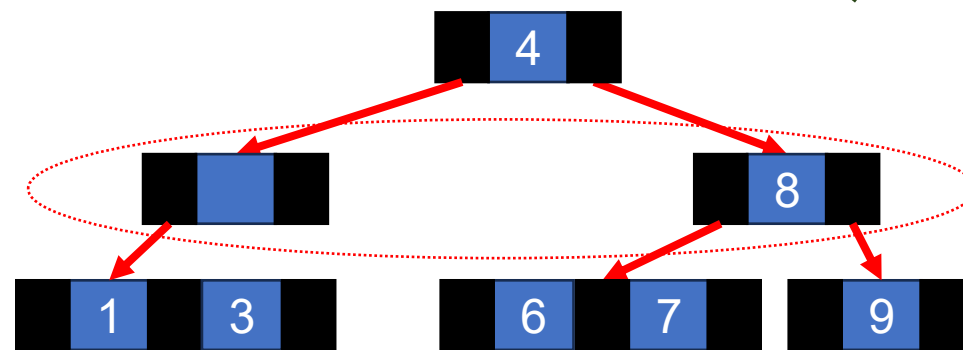
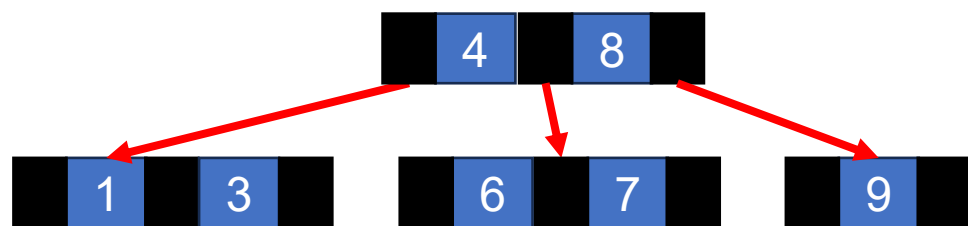
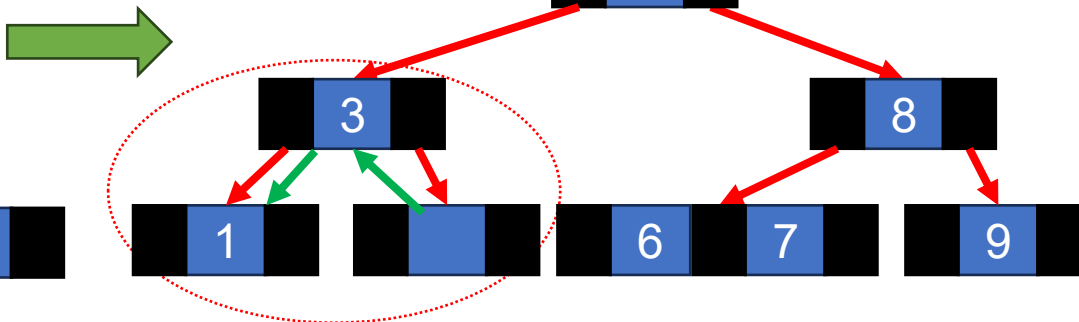
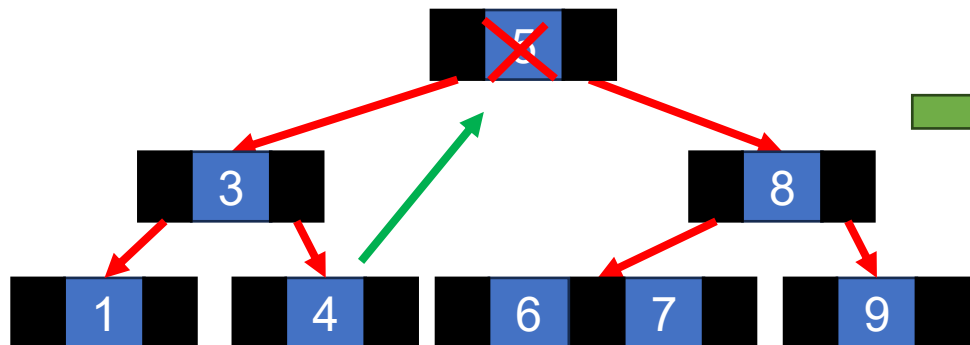


(α, b)-tree



Пример 1. Удаление 4 из (2,4) - дерева

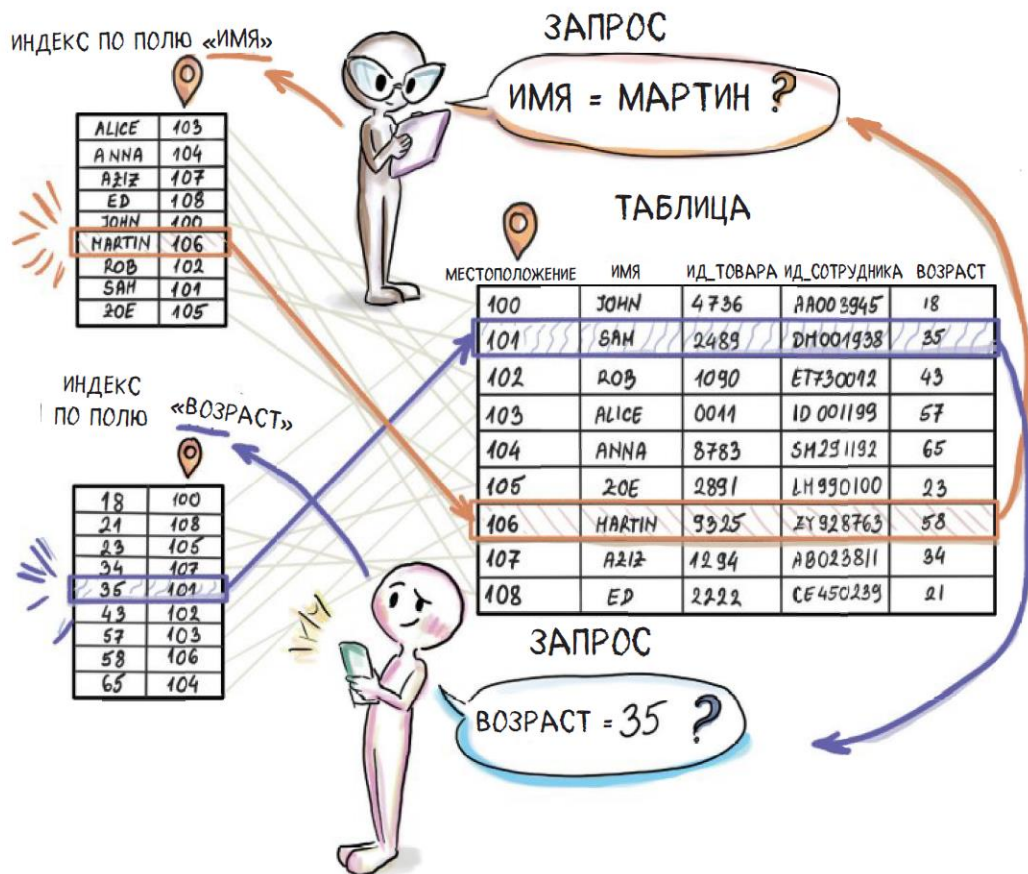
Пример 2. Удаление 5 из (2,4) - дерева



Прежде чем перейти к
деревьям немного
актуальности

Принцип работы индексации

Индекс наиболее полезен, когда он строится на основе столбца, к которому мы часто обращаемся с запросами. В ответе на отправляемый запрос, возможно, потребуется вернуть всю строку, в которой ключ совпадает с ключом запроса, но для локализации записи нужен только ключ – то есть значение из обозначенного индексного столбца.



Как вы понимаете, наличие одного индекса позволяет ускорять поиск по одному столбцу, но он совершенно бесполезен при поиске по другим критериям.

В-деревья составляют основу большинства наиболее популярных движков баз данных, таких как [PostgreSQL](#) и [MySQL](#).

Вε-дерево – это альтернативная структура данных, лежащая в основе систем хранения данных, таких как TokudB, которые стали популярными благодаря своей превосходной производительности вставки/удаления.

LSM-деревья – это структуры данных, лежащие в основе LevelDB, RocksDB, Cassandra и некоторых других движков, заботящихся только о высокопроизводительных вставках, которые выполняются быстрее, чем даже те, что находятся в Вε-деревьях.

СУБД – зачем?

Система управления базами данных

СУБД служат различным целям:

- для временных «горячих» данных,
- в качестве долговременного хранилища «холодных» данных,
- для сложных аналитических запросов,
- для получения доступа к значениям только по ключу,
- для хранения данных временных рядов,
- для хранения больших двоичных объектов.

СУБД часто разделяют на три основные категории:

- Базы данных для обработки транзакций в реальном времени (online transaction processing, OLTP)
- Базы данных для аналитической обработки данных в реальном времени (online analytical processing, OLAP)
- Гибридная транзакционно-аналитическая обработка (hybrid transactional and analytical processing, HTAP)

Существует и множество других терминов и способов классификации:

- хранилища типа «ключ–значение»,
- реляционные базы данных,
- документоориентированные хранилища
- графовые базы данных.



Архитектура СУБД

Не существует общего шаблона для проектирования СУБД. Каждая база данных строится немного по-разному, а границы компонентов довольно трудно обнаружить и определить.

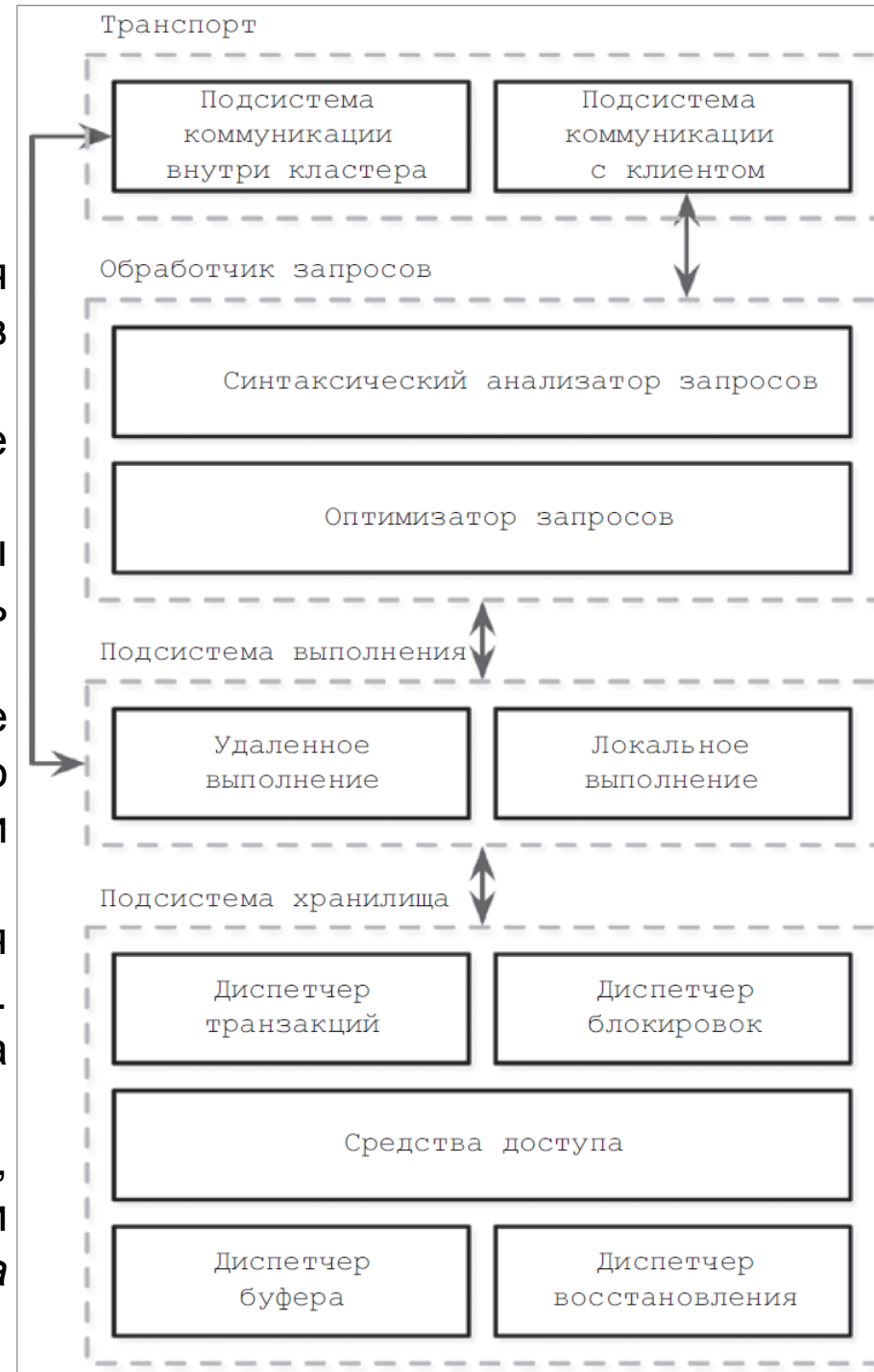
Архитектура, показанная на рисунке, демонстрирует некоторые общие элементы этих представлений.

СУБД используют *модель «клиент — сервер»*, в которой экземпляры СУБД (*узлы*) играют роль серверов, а экземпляры приложений — роль клиентов.

Запросы клиентов поступают через *транспортную подсистему*. После получения запроса транспортная подсистема передает его *обработчику запросов*, который анализирует, интерпретирует и проверяет его. Запрос обычно представлен в виде *плана выполнения*.

План выполнения обрабатывается *подсистемой выполнения*, которая собирает результаты выполнения локальных и удаленных операций. *Удаленное выполнение* обычно сводится к записи и чтению данных на других узлах кластера, а также выполнению репликации.

Локальные запросы выполняются *подсистемой хранения данных*, которая включает в себя несколько компонентов с четко заданными функциями: *Диспетчер транзакций*, *Диспетчер блокировок*, *Средства доступа*, *Диспетчер буфера*, *Диспетчер восстановления*.



Резидентные и дисковые СУБД

СУБД хранят данные в оперативной памяти или на диске.

Резидентные СУБД (или *СУБД в оперативной памяти*) хранят данные *главным образом* в оперативной памяти, а диск используют для восстановления и ведения журнала.

Дисковые СУБД хранят *большую часть* данных на диске и используют память для кэширования содержимого диска или в качестве временного хранилища

Резидентные СУБД отличаются от дисковых не только основной средой хранения данных, но и тем, как они организованы и какие структуры данных и методы оптимизации они используют.

Основными ограничивающими факторами роста количества резидентных баз данных являются непостоянство оперативной памяти и высокая стоимость. Так как содержимое ОЗУ является непостоянным, программные ошибки, отказы, аппаратные сбои и перебои в подаче электроэнергии могут привести к потере данных.

Ситуация, вероятно, будет меняться по мере роста доступности и популярности технологии энергонезависимой памяти (Non-Volatile Memory, NVM).

Записи журнала обычно применяются к резервной копии пакетами. После обработки пакета записей журнала резервная копия содержит *моментальный снимок* базы данных, который соответствует некоторому моменту, и все предшествующее содержимое журнала может быть удалено. Этот процесс называется *созданием контрольных точек*.

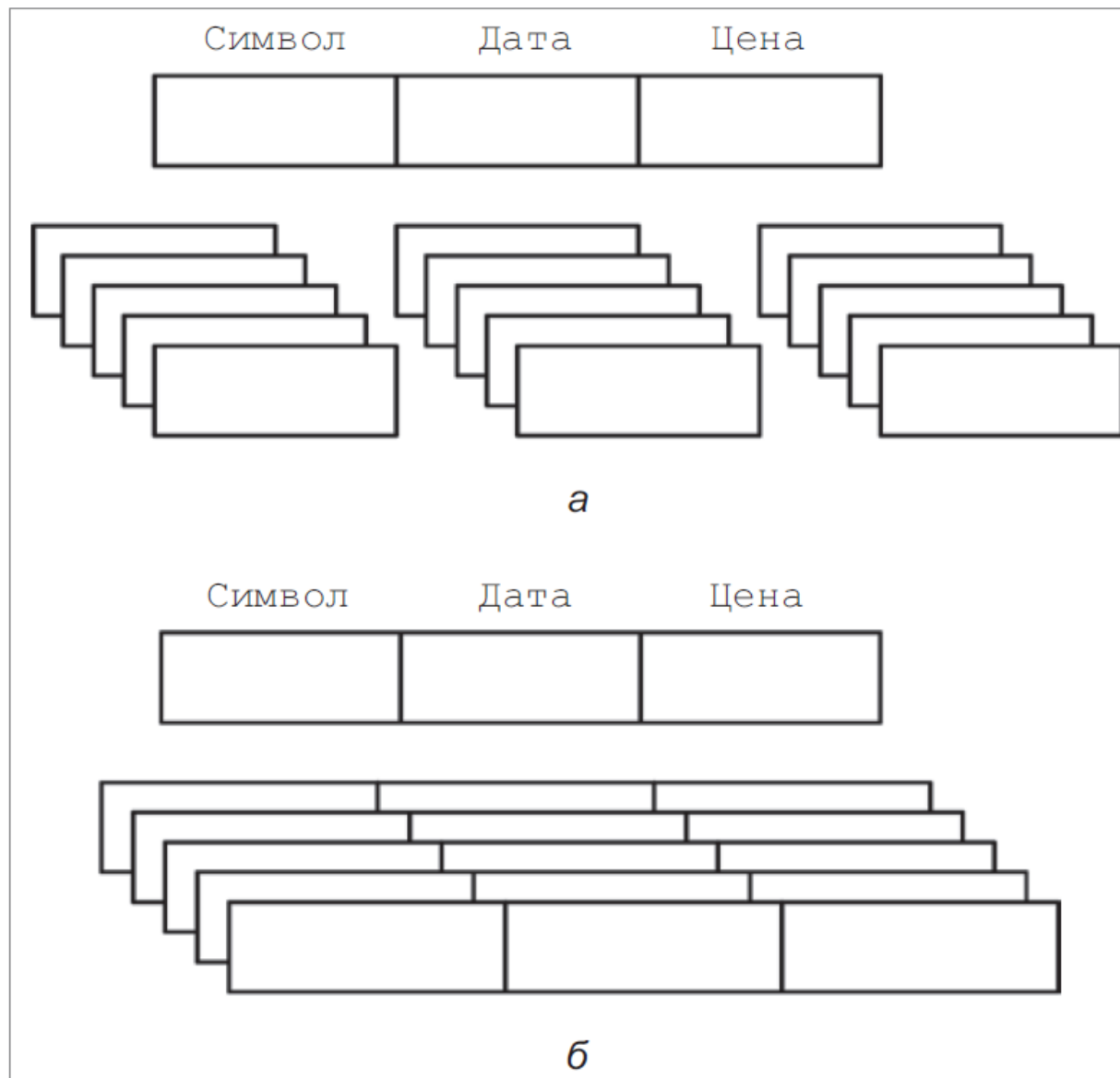


Колоночные и строчные СУБД

Большинство СУБД хранит некоторый *набор записей*, состоящий из *столбцов* и *строк таблиц*. Поле находится на пересечении столбца и строки и содержит одно значение некоторого типа. Поля, относящиеся **к одному столбцу, обычно имеют один и тот же тип данных**.

Один из способов классификации баз данных сводится к их разделению в зависимости от того, **как данные сохраняются на диске**: по строкам или по столбцам. Данные таблиц могут разбиваться либо по горизонтали (когда вместе сохраняются значения, относящиеся к одной строке), либо по вертикали (когда вместе сохраняются значения, относящиеся к одному столбцу). Разница между этими способами показана на рисунке: (а) значения разбиваются по столбцам и (б) значения разбиваются по строкам.

Примеров строчных СУБД очень много: [MySQL](#), [PostgreSQL](#) и др. Первыми примерами колоночных хранилищ стали свободно распространяемые СУБД [MonetDB](#) и [C-Store](#) (на основе C-Store - СУБД [Vertica](#)).



Колоночные и строчные СУБД

Колоночные хранилища хорошо подходят для аналитической обработки, требующей вычисления агрегатных величин, — для определения тенденций, вычислении средних значений и т. д.

Идентификатор	Символ	Дата	Цена
1	DOW	08 августа 2018	24314,65
2	DOW	09 августа 2018	24136,16
3	S&P	08 августа 2018	2414,45
4	S&P	09 августа 2018	2232,32

Символ: 1:DOW; 2:DOW; 3:S&P; 4:S&P

Дата: 1:08 авг 2018; 2:09 авг 2018; 3:08 авг 2018; 4:09 авг 2018

Цена: 1:24 314,65; 2:24 136,16; 3:2 414,45; 4:2 232,32

Для восстановления кортежей данных, которые могут быть полезны для объединения, фильтрации и многорядных агрегатов, нам необходимо сохранить некоторые метаданные на уровне столбцов, чтобы определить, с какими элементами данных из других столбцов они связаны.

В течение последних нескольких лет, вероятно, из-за растущего спроса на выполнение сложных аналитических запросов по увеличивающимся наборам данных появилось много новых колоночных форматов файлов, таких как [Apache Parquet](#), [Apache ORC](#), [RCFile](#), а также много колоночных хранилищ, таких как [Apache Kudu](#), [ClickHouse](#) и многие другие.



Хранилища с широкими столбцами

Колоночные базы данных не следует путать с хранилищами с широкими столбцами, такими как [BigTable](#) или [HBase](#), где данные представлены в виде многомерной карты, столбцы сгруппированы в семейства столбцов (обычно хранящие данные одного типа), а внутри каждого семейства столбцов данные сохраняются построчно. Такая структура лучше всего подходит для хранения данных, извлекаемых с помощью ключа или последовательности ключей.

Классический пример — Webtable сохраняет моментальные снимки содержимого веб-страниц, их атрибутов и связей между ними в момент, соответствующий определенной временной метке. Концептуальное представление структуры Webtable:

```
"com.cnn.www": {  
  contents: {  
    t6: html: "<html>..."  
    t5: html: "<html>..."  
    t3: html: "<html>..."  
  }  
  anchor: {  
    t9: cnnsi.com: "CNN"  
    t8: my.look.ca: "CNN.com"  
  }  
}  
"com.example.www": {  
  contents: {  
    t5: html: "<html>..."  
  }  
  anchor: {}  
}
```

Файлы данных и индексные файлы

Основной целью СУБД являются хранение данных и обеспечение быстрого доступа к ним. Но как организованы эти данные? Почему нужна именно СУБД, а не просто некоторый набор файлов? Как организация файлов повышает эффективность работы?

СУБД используют файлы для хранения данных, но вместо того, чтобы для поиска записей

полагаться на иерархию каталогов и файлов файловой системы, они формируют файлы с использованием специальных форматов, зависящих от конкретной реализации системы. Основные причины использования специальной структуры файлов вместо неструктурированных файлов:

- Эффективность хранения
- Эффективность доступа
- Эффективность обновления

СУБД хранят *записи данных*, состоящие из нескольких полей, в таблицах, а каждая таблица обычно представлена отдельным файлом. Каждую запись в таблице можно найти с помощью *ключа поиска*. Для поиска записей СУБД используют *индексы*: вспомогательные структуры данных, которые позволяют эффективно находить записи данных без сканирования всей таблицы.

Семейство столбцов: содержимое

Ключ строки	Временная метка	Квалификатор	Значение
com.cnn.www	t3	html	"<html>..."
com.cnn.www	t5	html	"<html>..."
com.cnn.www	t6	html	"<html>..."
com.example.www	t5	html	"<html>..."

Семейство столбцов: якорь

Ключ строки	Временная метка	Квалификатор	Значение
com.cnn.www	t8	cnnsi.com	"CNN"
com.cnn.www	t5	my.look.ca	"CNN.com"

Файлы данных и индексные файлы

Файлы данных

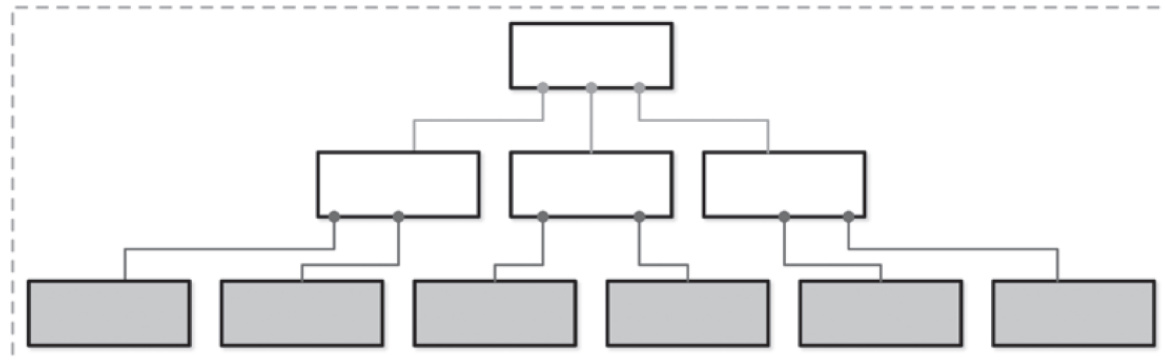
Файлы данных (иногда называемые *первичными* файлами) могут быть реализованы в виде *индексированной таблицы* (index-organized table, IOT), *таблицы-кучи* (файла-кучи) или *хэшированной таблицы* (хэшированного файла).

Когда записи хранятся в отдельном файле, индексные файлы содержат *элементы данных*, однозначно идентифицирующие записи данных и содержащие достаточно информации, чтобы найти их в файле данных.

Индексные файлы

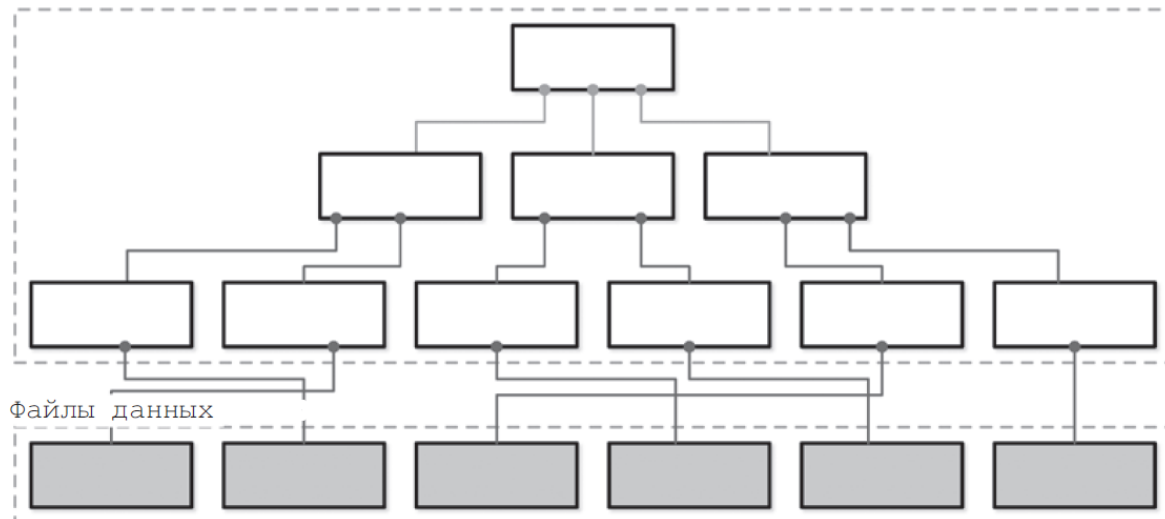
Индекс — это структура, с помощью которой записи данных организуются на диске таким образом, чтобы повысить эффективность операций извлечения. Индексные файлы организованы как специализированные структуры, которые сопоставляют ключи с теми местами в файлах данных, где хранятся записи, идентифицированные этими ключами (в случае файлов-куч) или первичными ключами (в случае индексированных таблиц). Индекс *первичного файла* (файла данных) называется *первичным индексом*.

Индексированная таблица



a

Индексный файл

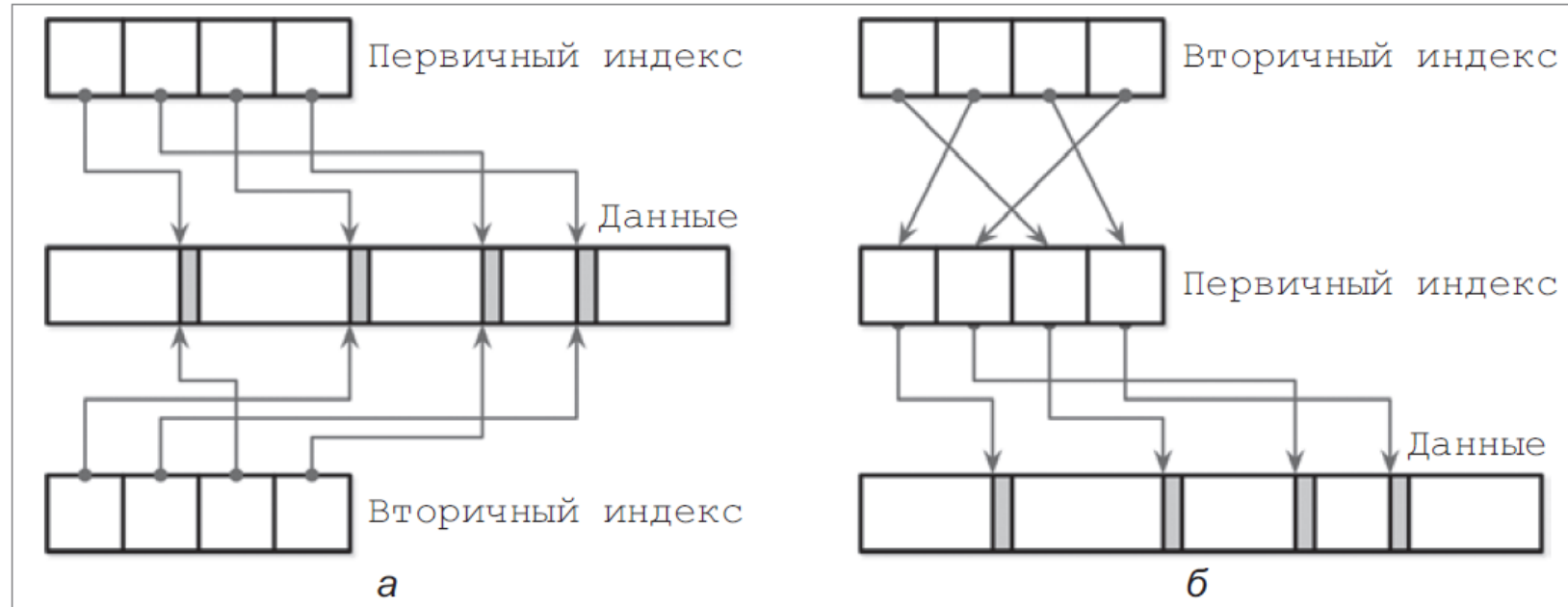


Файлы данных

б

Буферизация, неизменяемость и упорядочение

Каждая подсистема хранения построена на основе некоторой структуры данных. Однако эти структуры не описывают суть кэширования, восстановления, управления транзакциями и других элементов, надстраиваемых подсистемами хранения поверх этих структур.



Структуры хранения имеют три общие переменные:

- **Буферизация** Определяет, будет ли структура хранения накапливать определенный объем данных в памяти перед их размещением на диске.
- **Изменяемость (или неизменяемость)** Определяет, будет ли структура хранения после считывания частей файла и их обновления записывать обновленные результаты в то же место в файле. Неизменяемые структуры доступны только для добавления: после записи содержимое файла не изменяется. Вместо этого изменения добавляются в конец файла.
- **Упорядочение** Определяется тем, совпадает ли порядок (сортировка) записей данных в страницах на диске с порядком ключей в данных. Хранятся ли соседние ключи в смежных сегментах на диске?

Форматы файлов

Как именно В-деревья и другие структуры реализуются на диске? Доступ к диску отличается от доступа к оперативной памяти: с точки зрения разработчика приложений, доступ к памяти почти всегда прозрачен. Благодаря виртуальной памяти нам не нужно рассчитывать смещения для указателей вручную. Доступ к дискам осуществляется с помощью СИСТЕМНЫХ ВЫЗОВОВ. Как правило, нам необходимо указать смещение (адрес) внутри целевого файла, а затем перевести дисковое представление в форму, подходящую для оперативной памяти.

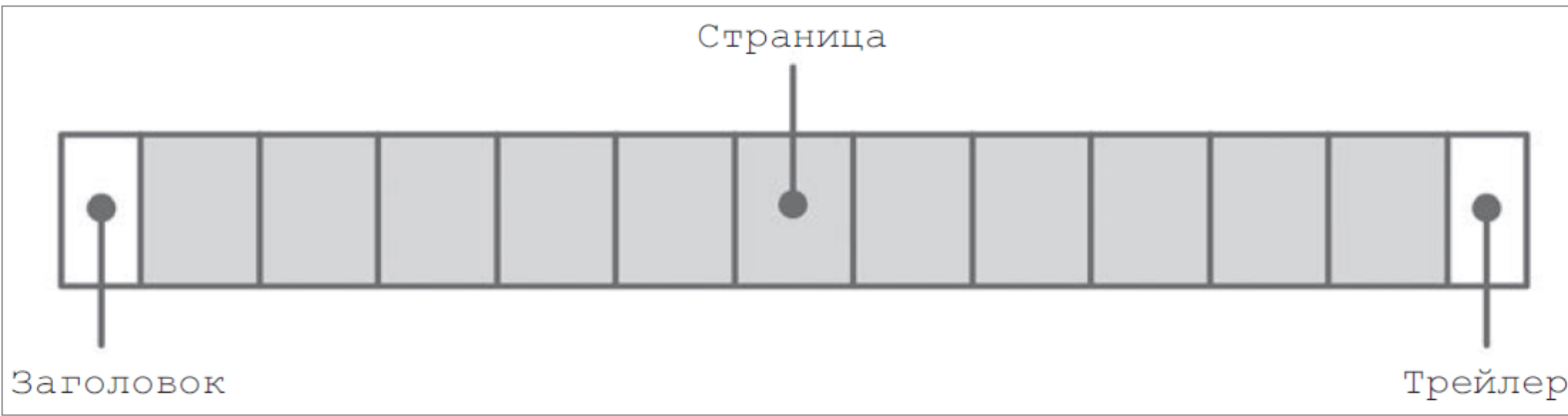
Это означает, что для достижения эффективности при разработке дисковых структур необходимо учитывать различия между этими формами. Для этого мы должны придумать формат файла, который было бы легко формировать, изменять и интерпретировать.

Семантика управления указателями в дисковых структурах несколько отличается от семантики резидентных структур. Дисковые В-деревья можно рассматривать как некоторый механизм управления страницами: алгоритмы должны формировать страницы и обеспечивать навигацию по ним. Соответственно необходимо рассчитывать и размещать страницы и указатели на них.



Основные принципы

Обычно разработка формата файла начинается с решения о том, как будет осуществляться адресация: будет ли файл разделен на страницы одинакового размера, представленные одним блоком или несколькими смежными блоками. Большинство структур хранения с обновлением на месте используют страницы постоянного размера. Файл обычно начинается с заголовка фиксированного размера и может заканчиваться трейлером фиксированного размера, содержащим вспомогательную информацию, к которой необходимо обеспечить быстрый доступ или которая необходима для декодирования остальной части файла.



Во многих хранилищах, схема данных является фиксированной. Вы кажется читали заголовки wav, jpg, bmp... (посмотрите через hex)



Структура страницы

СУБД хранят записи в файлах данных и индексных файлах. Эти файлы разбиты на секции фиксированного размера, называемые **страницами**, размер которых часто составляет несколько блоков файловой системы. Размер страниц обычно находится в диапазоне от 4 до 16 килобайт.

Пример - узел дискового B-дерева

Каждый узел B-дерева занимает одну страницу или несколько связанных друг с другом страниц, поэтому в контексте B-деревьев термины узел и страница (и даже блок) часто используются как синонимы.

P_0	k_1	v_1	P_1	k_2	v_2	...	k_n	v_n	P_n	не использовано
-------	-------	-------	-------	-------	-------	-----	-------	-------	-------	-----------------

Способ организации страниц для записей фиксированного размера

Этот подход легко поддается реализации, однако недостатки:

- При добавлении ключа не справа, а в каком-либо другом месте потребуется перемещать элементы.
- Данный способ хорошо подходит только для данных фиксированного размера в противном случае не может обеспечить эффективное управление и доступ к данным.



Слоттированные страницы

При сохранении записей переменного размера **основной проблемой является управление свободным пространством**, т. е. высвобождение пространства, занятого удаляемыми записями.

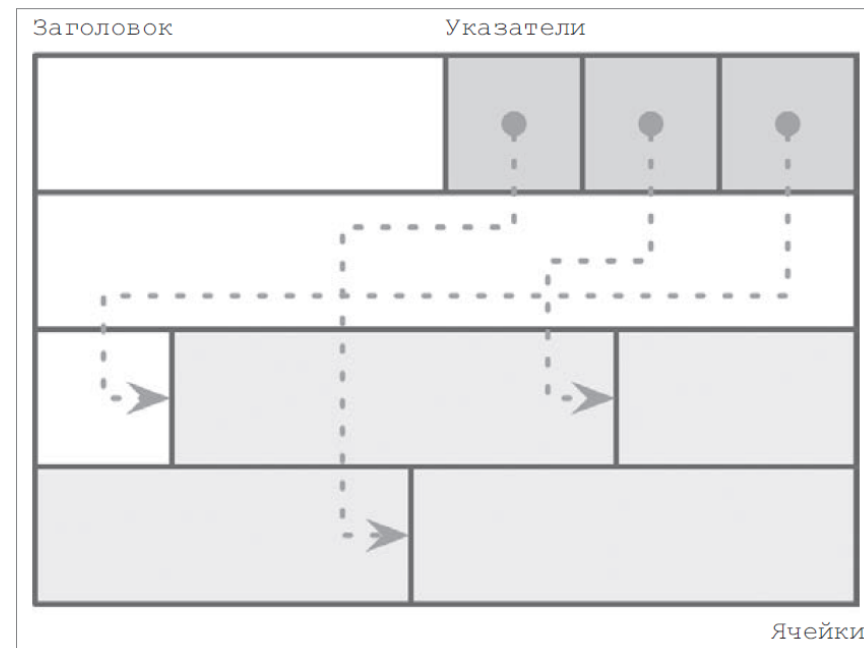
Чтобы упростить управление пространством в случае записей переменного размера, мы можем разделить страницу на сегменты фиксированного размера. Однако и такой подход в конечном итоге приведет к **потере пространства**. Нам нужен формат страницы, который позволяет:

- Сохранять записи переменного размера с минимальными накладными расходами.
- Высвобождать место, занятое удаляемыми записями.
- Ссылаться на записи страницы независимо от их точного расположения.

Для эффективного сохранения записей переменного размера, таких как строки, большие двоичные объекты и т. д., можно использовать такой способ организации, как **слоттированная страница**. Этот подход используется, например в PostgreSQL. При этом страница организуется в виде набора **слотов** или **ячеек** с отведением для указателей и ячеек двух независимых областей памяти, расположенных в разных частях страницы. Для сохранения порядка достаточно производить реорганизацию указателей.

Как такой подход устраняет проблемы, о которых мы говорили в начале этого раздела:

- Минимальные накладные расходы: накладные расходы — это массив указателей.
- Высвобождение пространства: пространство можно восстановить путем дефрагментации и перезаписи.
- Динамическая компоновка: за пределами страницы обращение к слотам производится только посредством идентификаторов, поэтому данные о точном местоположении скрыты внутри страницы.



Слоттированная страница

Структура ячеек

Используя флаги, перечисления и примитивные типы, мы можем начать проектировать структуру ячеек, а затем объединить ячейки в страницы и составить дерево из страниц. На уровне ячеек мы различаем ячейки, содержащие ключи, и ячейки, содержащие пары «ключ–значение». Ячейка с ключами содержит ключ-разделитель и указатель на страницу, расположенную *между* двумя соседними указателями. Для формирования ячейки с ключом нам нужно знать:

- Тип ячейки (который можно определить в метаданных самой страницы).
- Размер ключа.
- Идентификатор дочерней страницы, на которую указывает эта ячейка.
- Байты ключа.

Структура ячейки с ключом переменного размера может выглядеть примерно так:

```

0      4      8
┌──[int]──key_size──┐┌──[int]──page_id──┐┌──[bytes]──key──┐
└──────────────────┘└──────────────────┘└──────────────────┘
  
```

Ячейки с парами «ключ–значение» отличаются тем, что содержат записи данных вместо идентификаторов дочерних страниц. В остальном их структура аналогична:

- Тип ячейки (можно вывести логическим путем из метаданных страницы).

```

0 1 5 ...
┌──[byte]──flags──┐┌──[int]──key_size──┐
└──────────────────┘└──────────────────┘
  
```

- Размер ключа.
- Размер значения.
- Байты ключа.

```

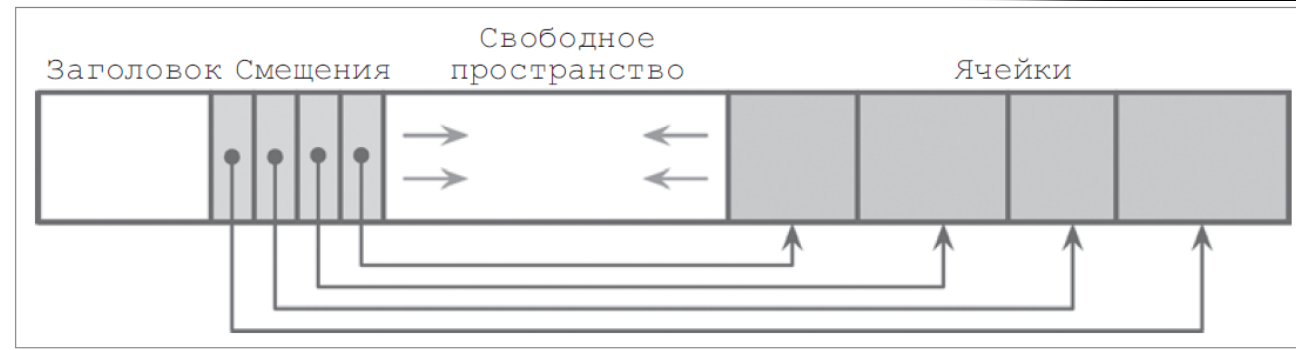
5 9 .. + key_size
┌──[int]──value_size──┐┌──[bytes]──key──┐┌──[bytes]──data_record──┐
└──────────────────┘└──────────────────┘└──────────────────┘
  
```

Чтобы сослаться на ячейку, нам достаточно указать ее *смещение* (offset).

Структура ячеек

Чтобы организовать ячейки в страницы, мы можем использовать метод слоттированных страниц. В этом случае ячейки добавляются в правую часть страницы (ближе к ее концу) с сохранением смещений/указателей ячеек в левой части страницы.

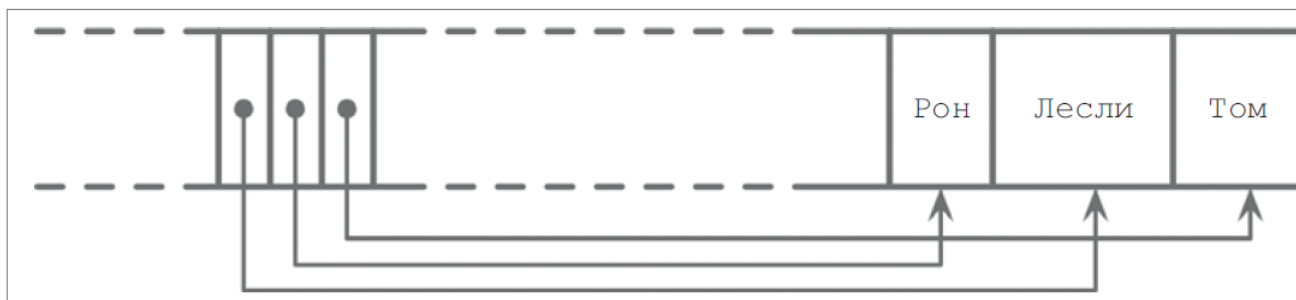
И значения при этом не обязательно вставлять по порядку. При вставке новые ячейки будут добавлены в свободное место, без реорганизации ячеек, а их логический порядок будет поддерживаться с помощью пересортировки их смещений. Такая структура позволяет добавлять ячейки на страницу с минимальными усилиями, так как ячейки не нужно перемещать во время операций вставки, обновления или удаления.



Смещение и направление роста ячеек

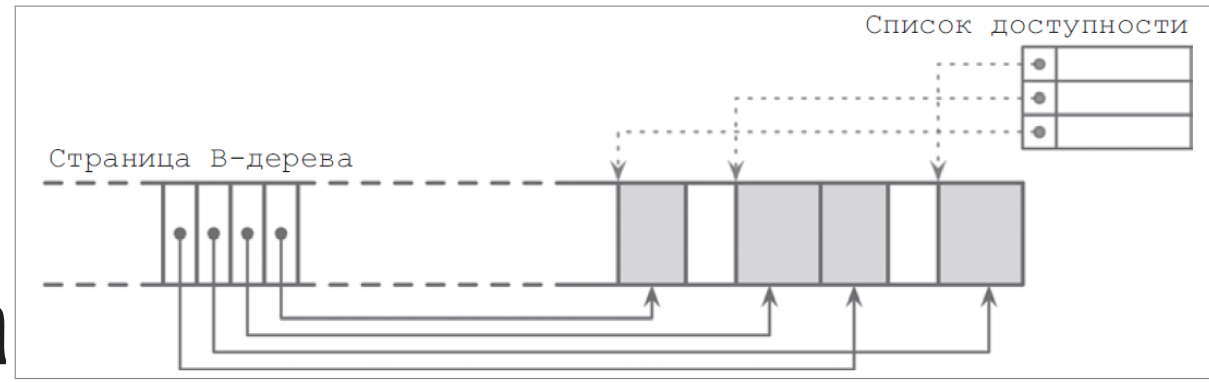


Записи, добавленные в произвольном порядке: «Том», «Лесли»



Добавление еще одной записи: «Рон»

Управление данными переменного размера



Фрагментированная страница и список доступности. Занятые страницы выделены серым цветом. Пунктирные линии представляют собой указатели на незанятые области памяти из списка доступности

Для удаления элемента на странице не требуется удалять саму ячейку и смещать другие ячейки для заполнения освободившегося пространства. Вместо этого можно пометить ячейку как удаленную и обновить размещенный в памяти список доступности, указав объем освободившейся памяти и предоставив указатель на освобожденную область.

В списке доступности хранятся смещения освобожденных сегментов и их размеры. Вставляя новую ячейку, мы сначала проверяем список доступности, чтобы определить, есть ли сегмент, в который она может поместиться.

В СУБД SQLite незанятые сегменты называются свободными блоками (freeblock), и указатель на первый свободный блок сохраняется в заголовке страницы. Кроме того, внутри страницы сохраняется общее количество доступных байтов, чтобы можно было быстро проверить, поместится ли новый элемент на странице после дефрагментации.

При поиске подходящей области памяти используются следующие стратегии:

- **Первый подходящий** Может привести к увеличению накладных расходов.
- **Наиболее подходящий** Для более эффективного использования пространства можно попытаться найти сегмент, при использовании которого останется наименьший остаток.

Управление версиями

СУБД постоянно развиваются, и разработчики работают над добавлением функций, исправлением ошибок и проблем с производительностью. В результате этого двоичный формат файла может измениться. В большинстве случаев любая версия подсистемы хранения должна поддерживать несколько форматов сериализации (например, текущий и один или несколько устаревших форматов для обратной совместимости). При этом у нас должна быть возможность узнать, с какой версией файла мы имеем дело.

Такую возможность можно обеспечить несколькими способами. Например, СУБД Apache Cassandra использует префиксы версий в именах файлов.

Кроме того, сведения о версии могут сохраняться в отдельном файле. Например, СУБД PostgreSQL сохраняет номер версии в файле PG_VERSION.

Сведения о версии также могут сохраняться непосредственно в заголовке индексного файла. В этом случае часть заголовка (или весь заголовок) должна кодироваться в формате, не меняющемся от версии к версии. Выяснив, для какой версии закодирован файл, мы можем создать рассчитанный на нее итератор для интерпретации содержимого. Некоторые форматы файлов идентифицируют версию с помощью системных кодов.



Вычисление контрольной суммы

Файлы на диске могут быть повреждены или искажены в результате программных ошибок и аппаратных сбоев. Чтобы сразу же выявлять эти проблемы, исключая распространение поврежденных данных в другие подсистемы или узлы, мы можем использовать контрольные суммы и циклический избыточный контроль (cyclic redundancy check, CRC).

Некоторые источники не делают различий между криптографическими и некриптографическими хэш-функциями, CRC и контрольными суммами. Все эти методы объединяет то, что они сводят значительный объем данных к некоторому небольшому числу, однако они различаются в части сценариев использования, назначения и предоставляемых гарантий.

Контрольные суммы дают самые слабые гарантии и не позволяют выявлять одновременное повреждение нескольких битов. Они обычно вычисляются путем использования исключающего ИЛИ в сочетании с проверкой четности или суммированием.

Циклический избыточный контроль (CRC) часто помогает выявлять пакеты ошибок (например, случаи повреждения нескольких последовательных битов)



Обработка транзакций и восстановление

Транзакция — это неделимая логическая единица работы в СУБД, позволяющая представить несколько операций в виде единого шага. Операции, выполняемые транзакциями, включают в себя чтение и запись значений базы данных. Транзакция базы данных должна обладать свойствами *атомарности*, *согласованности*, *изолированности* и *долговечности* (atomicity, consistency, isolation, durability, ACID).

Атомарность Стадии транзакции *неотделимы* друг от друга, т. е. либо все стадии, связанные с транзакцией, выполняются успешно, либо не выполняется ни одна из них.

Согласованность Согласованность — свойство, зависящее от приложения; транзакция должна переводить базу данных из одного допустимого состояния в другое допустимое состояние, сохраняя все инварианты базы данных (включая различные ограничения, ссылочную целостность и т. д.).

Изолированность Несколько параллельно выполняемых транзакций должны выполняться без взаимного влияния друг на друга, как если бы в это время не выполнялись никакие другие транзакции.

Долговечность После фиксации транзакции все изменения в состоянии базы данных должны персистентно храниться на диске и сохраняться в случае перебоев в электроснабжении и сбоев и отказов системы.

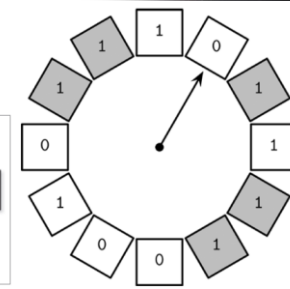
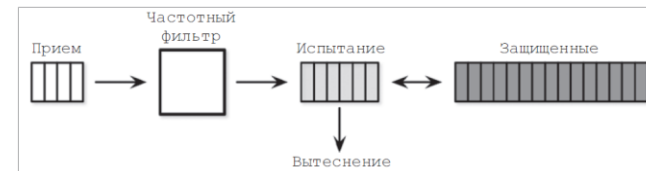
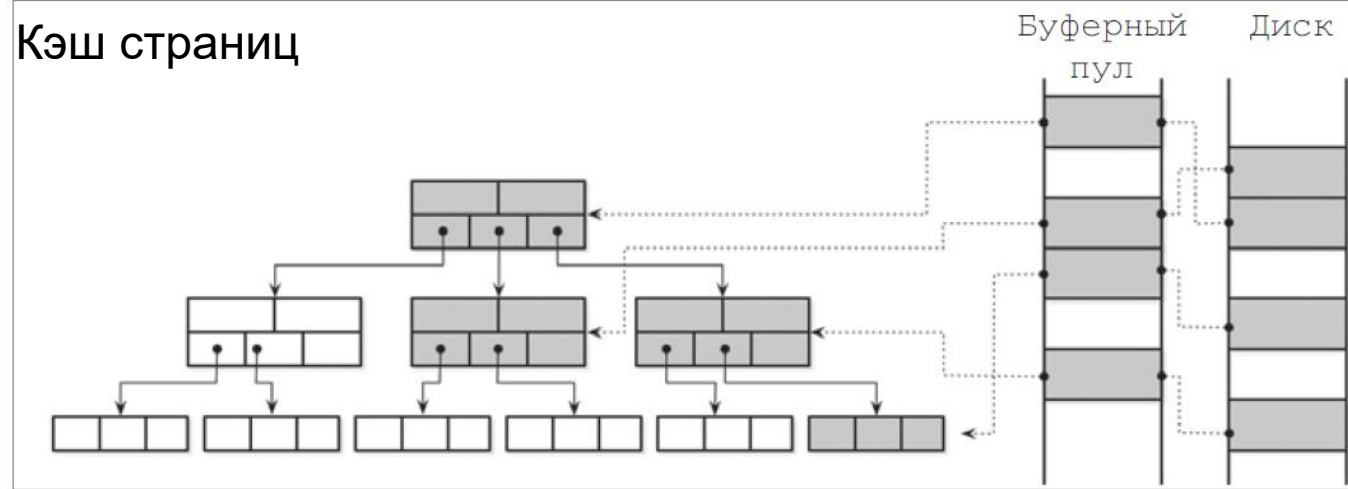


Организация буферизации данных

Большинство баз данных использует двухуровневую схему организации памяти, где одним уровнем является более медленный персистентный носитель (диск), а другим — более быстрая оперативная память (ОЗУ). Чтобы снизить количество обращений к персистентному носителю, страницы *кэшируются* в ОЗУ. При повторном запрашивании страницы уровнем хранения возвращается ее кэшированная копия.

Имеющиеся в памяти кэшированные страницы можно повторно использовать при условии, что другие процессы не будут вносить изменения в данные на диске. Такой подход иногда называют использованием *виртуального диска*. Более распространенное название этой концепции — *кэш страниц*, или *буферный пул*.

Кэширование страниц не ограничивается рамками баз данных. В операционных системах также есть понятие кэша страниц. Операционные системы используют *незанятые* сегменты памяти, чтобы прозрачно кэшировать содержимое диска для повышения производительности системных вызовов ввода-вывода. Процесс загрузки некашированных страниц в память называют *подкачкой страниц*. Если в кэшированную страницу вносятся изменения, она считается *грязной* до тех пор, пока эти изменения не будут *выгружены* (или *сброшены*) на диск.



Примеры: FIFO, LRU, [CLOCK](#), [LFU](#)

Восстановление

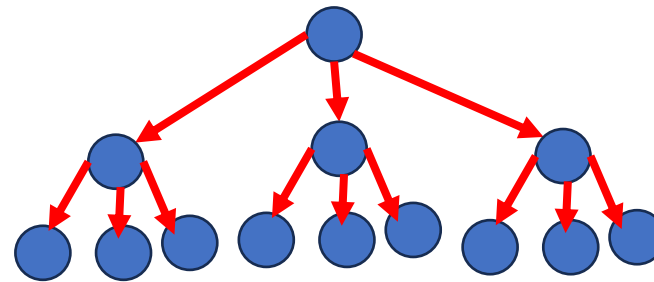
СУБД являются надстройкой поверх нескольких аппаратных и программных уровней, у которых могут быть свои проблемы со стабильностью и надежностью. При этом и сами СУБД, и лежащие в их основе программные и аппаратные компоненты могут иногда выходить из строя. Журнал упреждающей записи (write-ahead log, WAL), также иногда называемый журналом фиксации, — это вспомогательная дисковая структура с доступом только для добавления, которая используется для восстановления после сбоев и восстановления транзакций.

Основные функции журнала упреждающей записи можно свести к следующему:

- Позволяет кэшу страниц буферизировать обновления размещенных на диске страниц с обеспечением долговечности в общем контексте СУБД.
- Производит персистентное сохранение всех операций на диске до тех пор, пока кэшированные копии страниц, затронутых этими операциями, не будут синхронизированы с копиями на диске. Каждая операция, которая изменяет состояние базы данных, должна быть записана в журнале на диске до изменения содержимого соответствующих страниц.
- Позволяет в случае сбоя восстановить из журнала операций потерянные изменения, которые были произведены в памяти.
- В дополнение к этим функциям журнал упреждающей записи играет важную роль в обработке транзакций.

ARIES (Algorithm for Recovery and Isolation Exploiting Semantics, алгоритм восстановления и изоляции на основе семантики) — современный алгоритм восстановления, который широко используется

BufferTree



Статья про эти деревья [по ссылке](#). Основные операции:

$\text{insert}(k, t_s, v)$

$\text{remove}(k, t_s, \perp)$

$\text{lookup}(k, t_s, ?)$

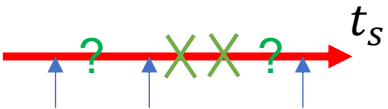
Операции должны быть глубоко буферизированы.

Очевидно асинхронность будет присутствовать.

Будет иметь ассимптотику: $O\left(\frac{1}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right)$ per item

Метка времени (Timestamp): Ключ, по которому данные упорядочиваются.

$\perp$ (Tombstone): Записи-метки, указывающие, что данные были удалены.



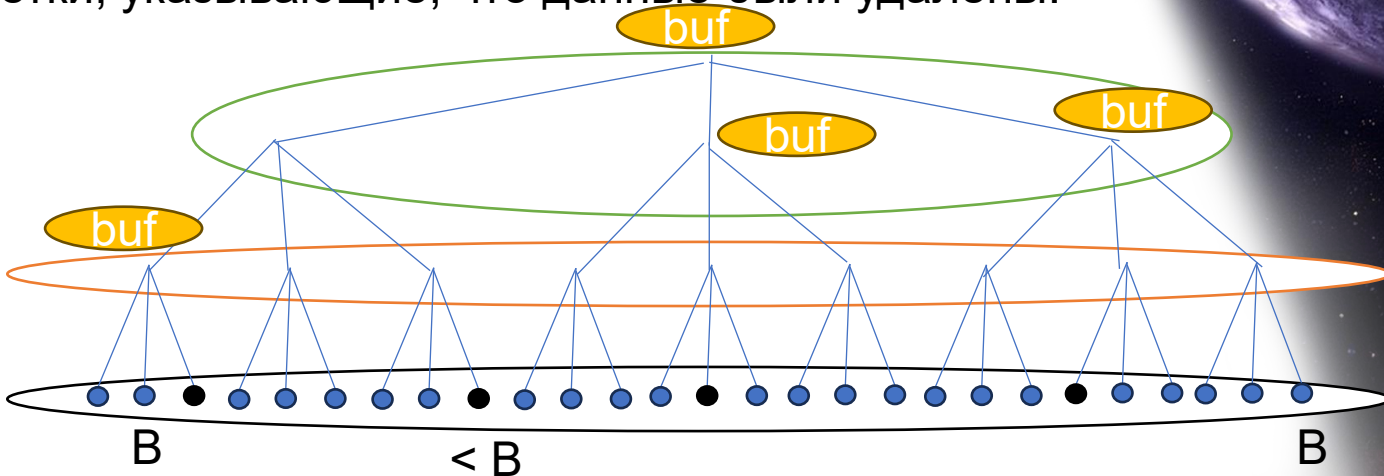
$$a \sim \frac{M}{B}, b = 4a$$

Пусть N – количество запросов, тогда количество вершин в дереве $N' \sim \frac{N}{B}$.

Другие слои

Предлистовый слой

Листовой слой



Блок – структура размером B . Из них состоят узлы, в листовом слое могут быть неполными. Данные, записанные в блоках идут в лексикографическом порядке.

Буфера – последовательности троек.

Ларс Ардже (Lars Arge) в 1996 год

BufferTree

Базово считаем, что буфер не переполняется.

$$\forall \text{ buffer} \leq M$$

Базовые действия. Корень дерева лежит в оперативной памяти. Когда происходит вставка, удаление и поиск происходят в верхний буфер.

Что делать, когда буфер переполнился?

$\text{buffer} \geq M$ – сразу сложно.

Если buffer переполняется:

1. Сортировка (доп. память $O(1)$)
2. Пролив (spill) вниз, распределение между сыновьями.

Пусть $x \geq M$. Операция Flush (запись с оперативки на жесткий) стоит $\sim \left\lceil \frac{x}{B} \right\rceil + b \sim \left\lceil \frac{x}{B} \right\rceil + \frac{M}{B}$

Для того чтобы 2ое слагаемое было небольшое – надо делать для Flush x как можно больше.

Когда все буфера почти полные – пока возможно переполнение...

Пусть буфер может быть больше чем M , но он может быть таким только во время flush.

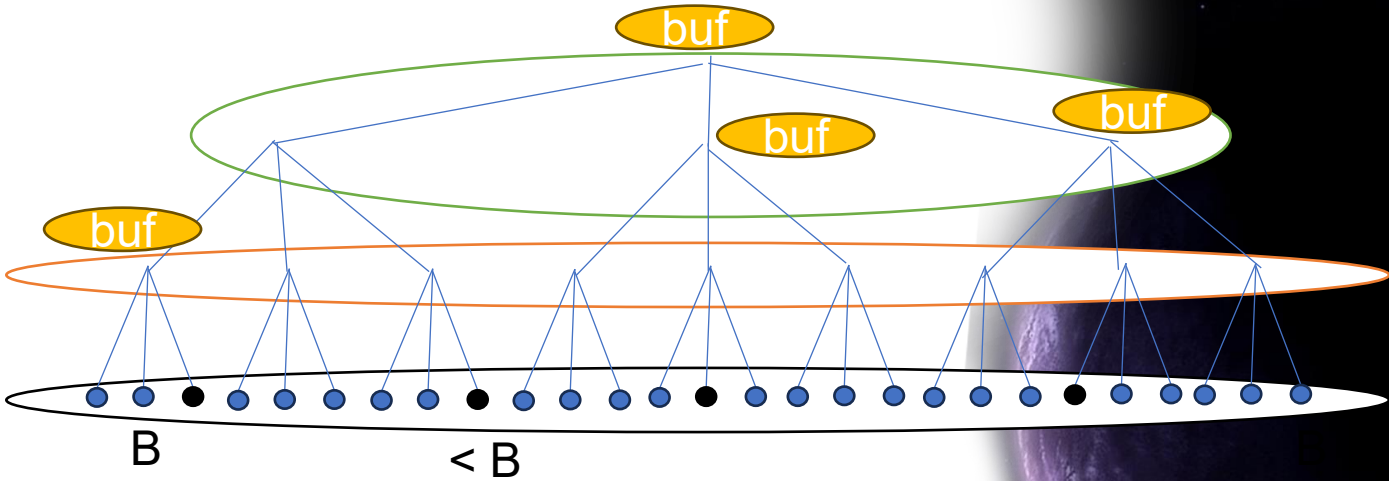


$< M$

Отсортирован
 $< (\log H - 1)M$

Данные в кружке и треугольнике – данные пришедшие от родителей от flush:

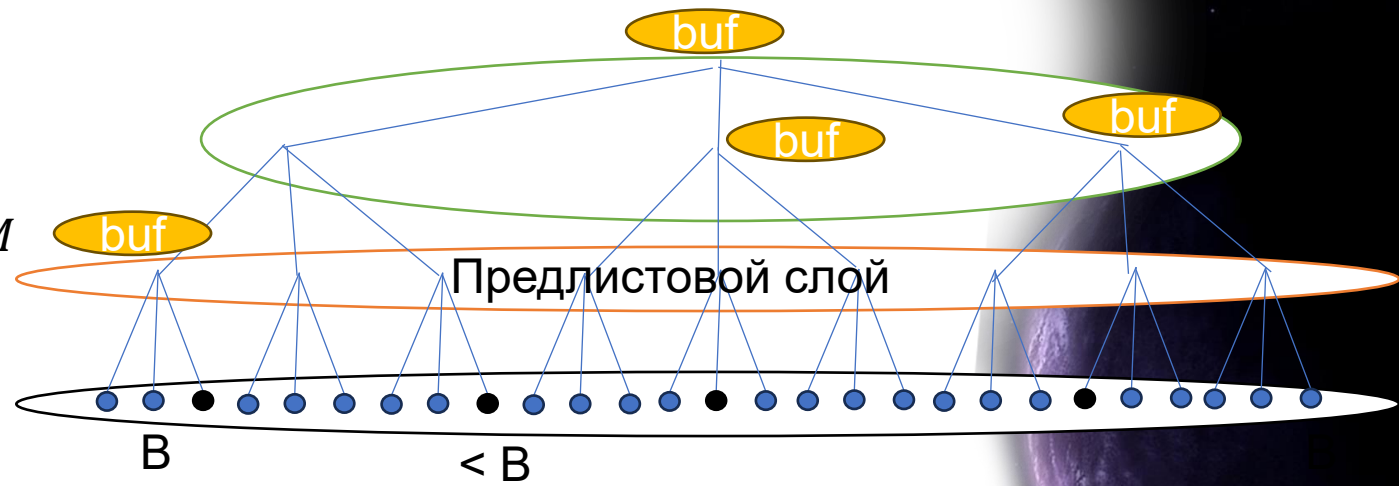
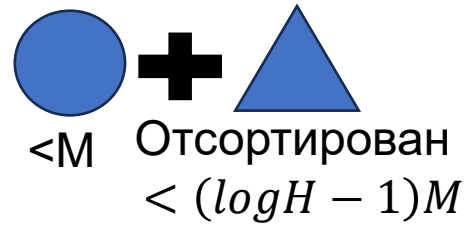
1. Если сумма данных $< M$ – забываем про треугольник
2. Если больше M – сортируем оперативку (кружок) и делаем слияние массив с треугольником



IO
Read
write

IO
worst case
every sun

BufferTree



Flush предлистового слоя

1. Разберемся с более высокими вершинами, перед **предлистовым** слоем
2. 3 потока – чтение кружок + треугольник + листво́вый слой
3. comraction – склеивание этих потоков – оставление вставки и удаление поиска (вся история есть)
4. Нарезка новых последовательностей блоков

Как нарезать на блоки $[a, b]$?

Если блоков мало – добавим фейковых (пустых) блоков.

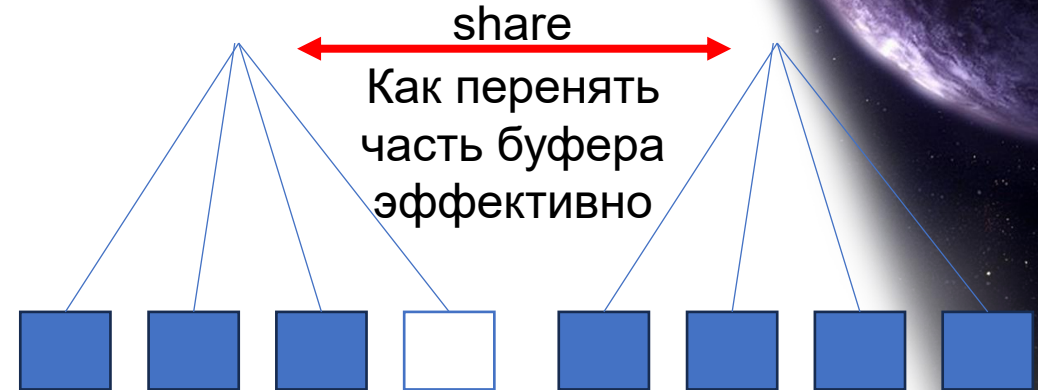
Если блоков слишком много

1. Положим b блоков – максимально возможное количество
2. Остальное – вставка в (a, b) дерево обычная

Если пустые блоки:

1. Операции удаления как в (a, b) дерево (fuse + share)
2. Решить проблемы с буферами.
 - а. Сделать flush этих буферов
 - б. Она может нет share так как split например произойдет и это будет уже другим узлом
 - в. Тогда вероятно сделать fuse (остальное д.з. (по желанию))

д.з. Показать, что нигде по скорости не выходим по времени.



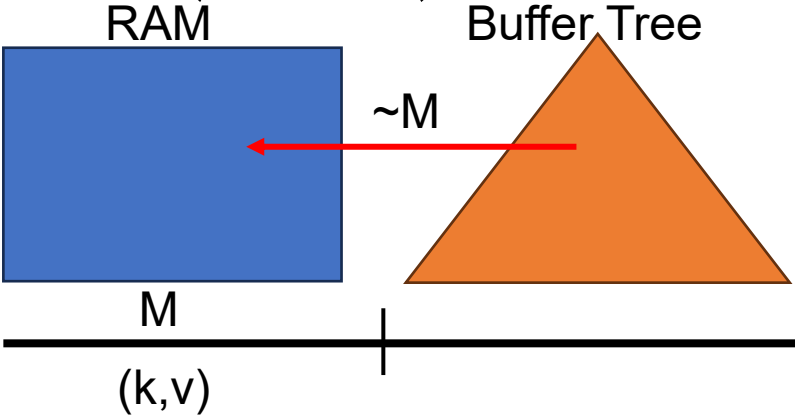
Heap

Хотим:

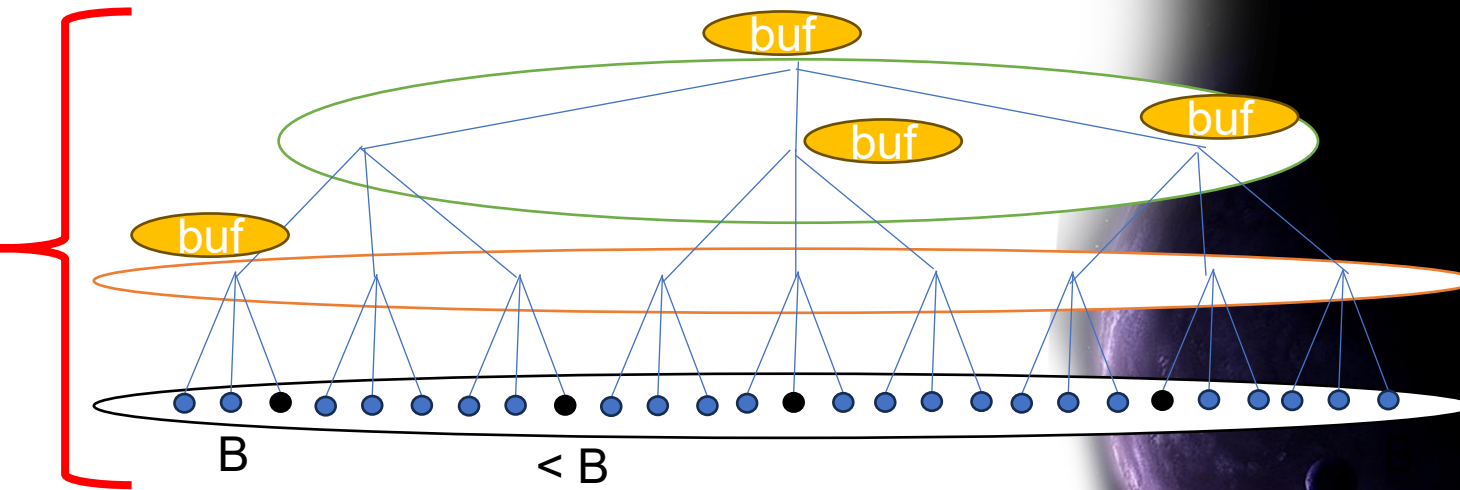
insert(k, v)

extract-min() → (k, v) – должно
давать ответ немедленно

Время $O\left(\frac{1}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right)$



$$H = \log_{\frac{M}{B}}\left(\frac{N}{B}\right)$$



insert(k, v)

Если ключ маленький – вставляем в оперативку, иначе
на диск

Если переполнение – spill в дерево.

Extract - RAM останется пустым - проблема

Будем делать flush(x) для пути до минимальных B (слева направо M/B штук)

Итого $\frac{(flush(x) \cdot H \cdot \frac{M}{B})}{M} = \frac{\left(\frac{1}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)\right) H \cdot \frac{M}{B}}{M} = \frac{1}{B} \log_{\frac{M}{B}}\left(\frac{N}{B}\right)$ - на 1 элемент (но это происходит редко, но в M раз дольше)

Повезло, что чтение по многу, но запись в дерево по 1 элементу.

B^ϵ - tree

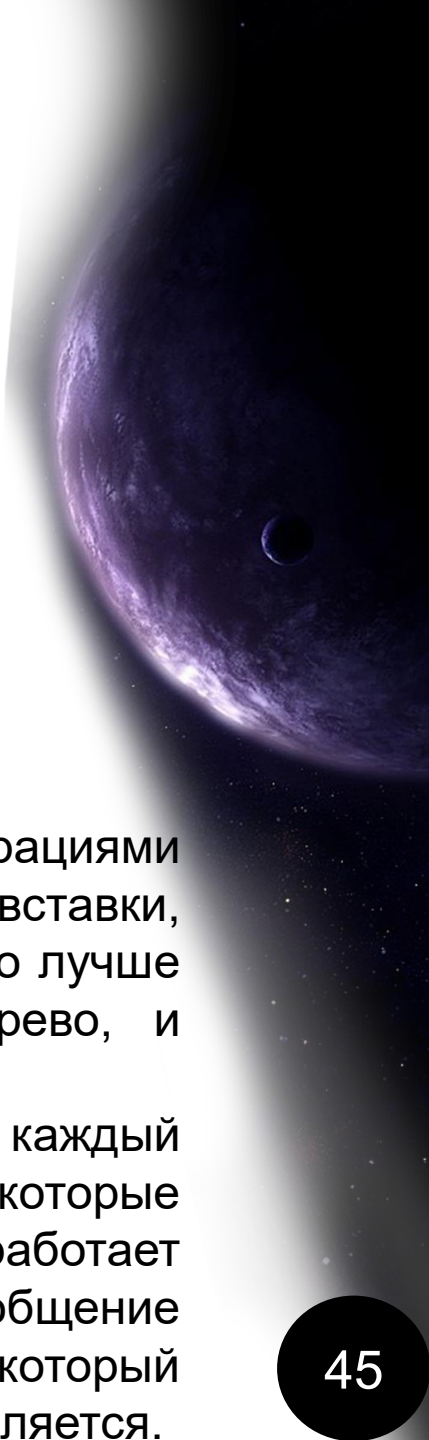
	B-tree	B^ϵ - tree
insert	$\log_f \frac{N}{B}$	$\frac{1}{B^{1-\epsilon} - 1} \frac{1}{\epsilon} \log_B \frac{N}{B}$
lookup	$\log_f \frac{N}{B}$	$\frac{1}{\epsilon} \log_B \frac{N}{B}$

B^ϵ -tree — это попытка получить лучшее из двух миров: B + BufferTree.

$\epsilon = \frac{1}{2}$ интересно тем, что в данной точке спектра мы получаем структуру данных с операциями поиска, которые хуже только на постоянный коэффициент, чем у B-деревьев, и операциями вставки, которые асимптотически лучше, чем у B-деревьев. Это означает, что B^ϵ -дерево значительно лучше подходит для рабочих нагрузок, оптимизированных под операции записи, чем B-дерево, и поддерживает асимптотически оптимальный поиск.

Ключевой конструкционной особенностью B^ϵ -деревьев является то, что, помимо ключей, каждый внутренний узел имеет буфер. Буферы призваны временно хранить вставки и удаления, которые действуют как сообщения на пути к назначенному листу. Операция удаления в B^ϵ -дереве работает не так, как в B-дереве. В буфер корневого узла первоначально вставляется сигнальное сообщение «Удалить x», и оно постепенно перемещается по буферам вниз по пути от корня к листу, который хранит x. Как только сигнальное сообщение достигает листа, содержащего x, x физически удаляется.

B^ϵ -дерево было разработано Бродалом (Brodal) и Фагербергом (Fagerberg) как структура данных, которая воплощает компромисс между скоростью операций вставки и поиска во внешней памяти. Компромисс отражается в диапазоне значений параметра $\epsilon = [0,1]$, который можно настраивать, и при $\epsilon = 0$ структура данных полностью оптимизирована под операции вставки/удаления; при $\epsilon = 1$ она полностью оптимизирована под операции поиска(это B-дерево).

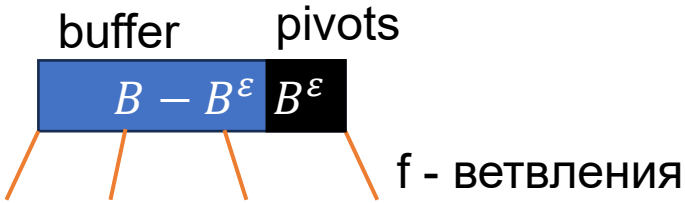


B^ϵ - tree

	B-tree	B^ϵ - tree
insert	$\log_f \frac{N}{B}$	$\frac{1}{B^{1-\epsilon} - 1} \frac{1}{\epsilon} \log_B \frac{N}{B}$
lookup	$\log_f \frac{N}{B}$	$\frac{1}{\epsilon} \log_B \frac{N}{B}$

Хотим сделать вставку быстрее, а поиск чуть медленнее.

Например, для логирования

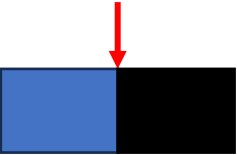


Пусть $0 < \epsilon < 1, f \sim B^\epsilon$

flush в буферизированном дереве очень сложен и плох в асимптотике

B^ϵ – хотим “не полный flush” но проще

- 1. Хотим посмотреть жирного сына и понять сколько хотим туда запихать $-\Delta = (B - B^\epsilon)/B^\epsilon$ (по принципу Дирихле)
- 2. У сына будет не более $B + \Delta$ ключей, снова пропихиваем Δ , и так до листьев
- 3. Листовой уровень – как и у обычного B дерева – новые листья, split и т. д., однако вместе с буфером



Вставка N операций раз в Δ вставок

$$\frac{B^\epsilon}{B - B^\epsilon} \frac{1}{\epsilon} \log_B \frac{N}{B}$$

Это лучше чем B дерево кратно в теории и немного на практике

LSM-деревья

Для того чтобы понять, как появились LSM-деревья, давайте начнем с самостоятельной разработки простой структуры данных, оптимизированной под операции записи. Каков оптимальный способ реализации индекса во внешней памяти с невероятно быстрой вставкой/удалением не прямо на месте, без учета скорости поиска?

На ум приходит простая регистрация сообщений о вставках/удалениях в одном последовательном журнале. Одним из вариантов такой структуры данных является резидентный буфер, в котором накапливаются сообщения о вставках, удалениях или модификациях записей. Как только буфер заполняется, мы последовательно сбрасываем его на диск. Затем мы снова заполняем резидентный буфер операциями записи (говоря «операции записи», мы подразумеваем операции вставки, удаления и модификации) и сбрасываем содержимое памяти, добавляя новые данные в конец журнала.

Описанная выше простая система гарантирует идеальную производительность вставки/удаления, равную $1/B$ на элемент.

Теперь попробуем немного модифицировать эту идею без ущерба для производительности записи. Допустим, что при каждом заполнении резидентного буфера все элементы в резидентном буфере внутренне сортируются. Буфер может представлять - сбалансированное двоичное дерево и сбрасывать отсортированный диапазон в отдельный файл или таблицу на диске. Мы имеем чуть более организованную систему, содержащую множество отдельных таблиц с данными, причем каждая таблица внутренне упорядочена. Если размер таблицы аналогичен размеру основной памяти M , а общее число обновлений равно N , то всего у нас будет N/M таблиц.

LSM-деревья

Поскольку каждая таблица отсортирована, мы можем использовать двоичный поиск, чтобы управлять поиском внутри отдельных таблиц, что помогает избегать полного линейного сканирования таблицы.

В результате мы получаем $O\left(\frac{N}{M} \times \log_2\left(\frac{M}{B}\right)\right)$ операций ввода-вывода в стоимости в расчете на запрос.

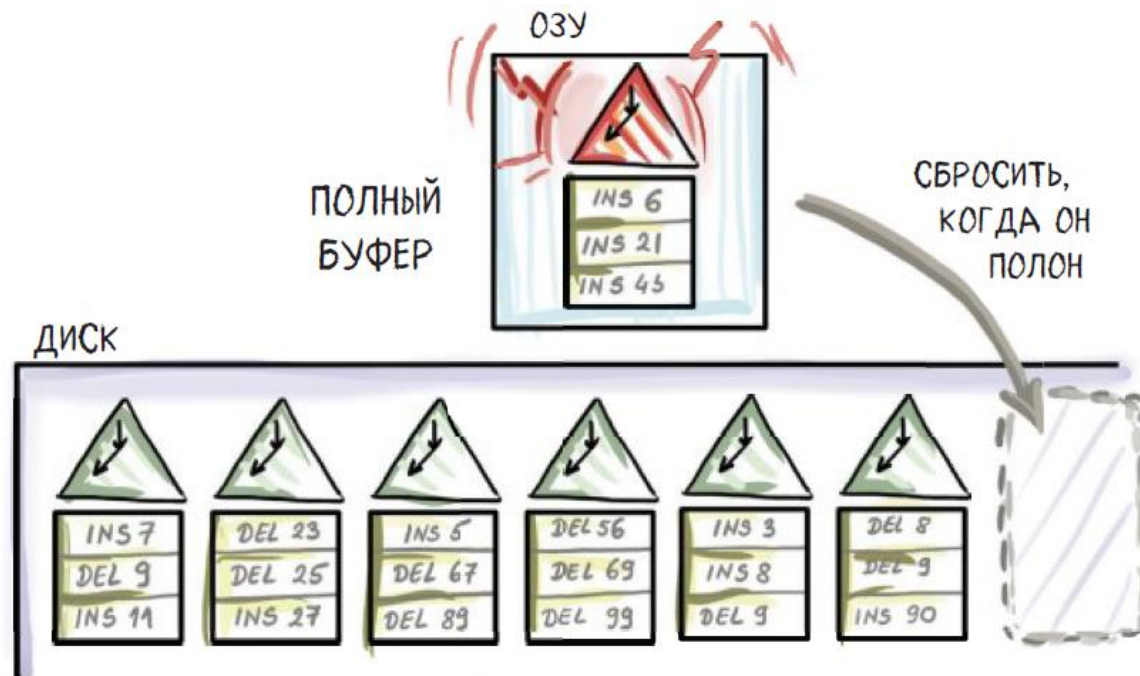
Давайте немного усовершенствуем нашу простую конструкцию: учитывая, что таблицы немутуируемы (мы не будем изменять их после сброса на диск), почему бы не построить индекс с использованием В+-дерева поверх каждой таблицы и не повысить производительность запросов до $O\left(\frac{N}{M} \times \log_B\left(\frac{M}{B}\right)\right)$ операций ввода-вывода?

Со временем растущее число таблиц усугубит и без того плохую производительность поиска.

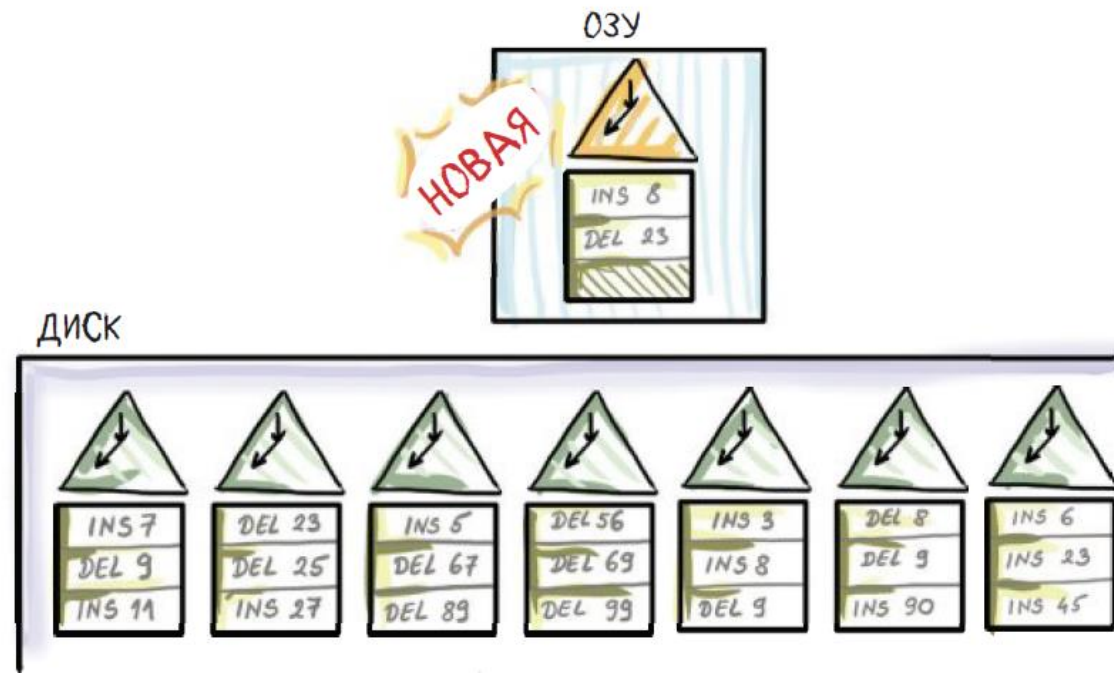
Поддержание нескольких фильтров Блума в оперативной памяти и топ!!!!!!!

Простая структура данных, оптимизированная под операции записи, но не являющаяся LMS-деревом

a)



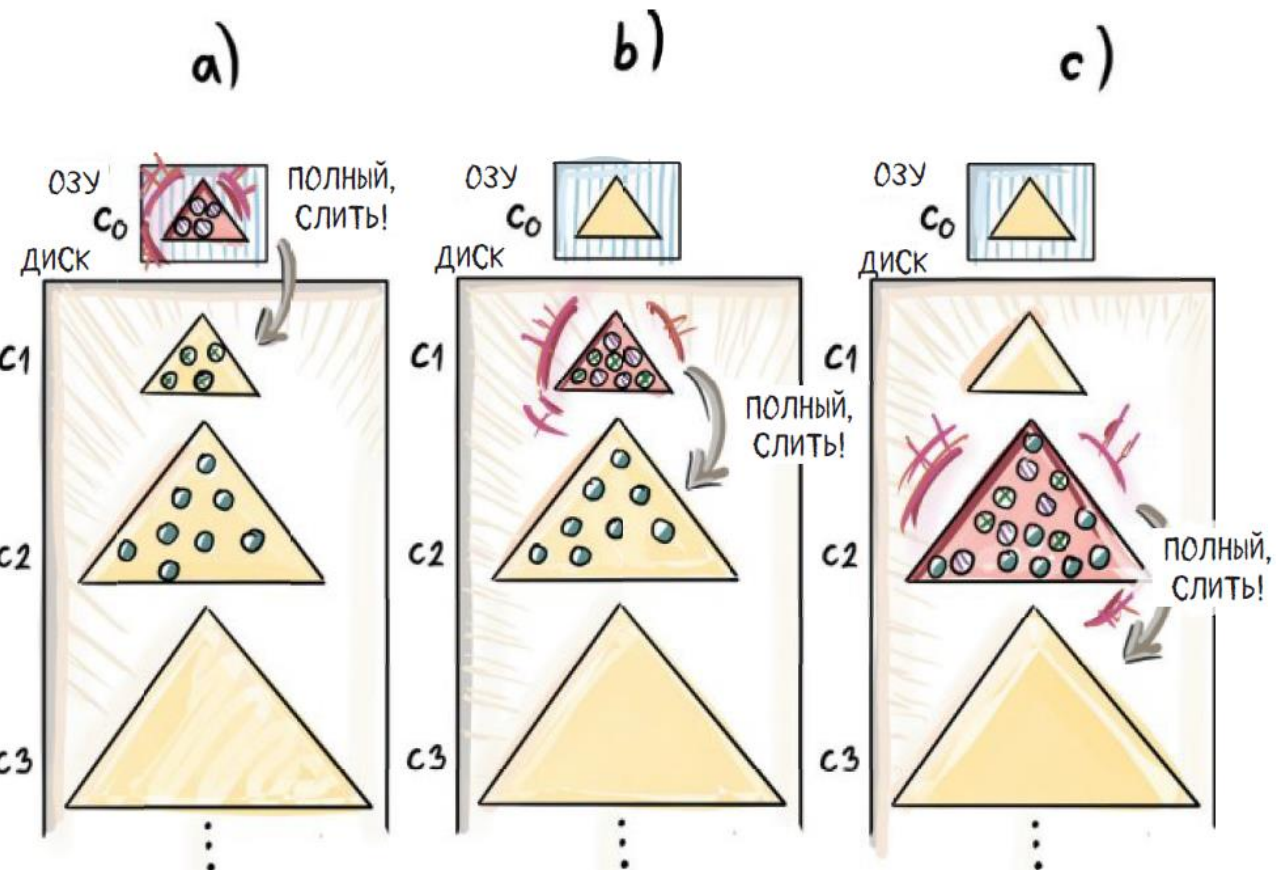
b)



LSM-деревья

Существует несколько вариантов базового устройства LSM-дерева, и множество реализаций. Первоначально LSM-дерево составлялось из k компонентов $C_0, C_1, \dots, C_{k-1}$, где C_0 находится во внутренней памяти, а все остальные на диске. Однако есть несколько важных отличий от нашей упрощенной структуры данных: в LSM-дереве мы исходим из того, что размер C_0 примерно равен размеру M памяти, а C_1 на f (обычно $f \geq 2$) больше компонента C_0 . На самом деле соотношение f поддерживается между размерами любых двух поочередных компонентов, поэтому размеры компонентов в возрастающем порядке равны M, fM, f^2M и т. д.

Экспоненциальное увеличение размеров между компонентами гарантирует, что в дальнейшем мы будем иметь управляемое число опрашиваемых компонентов. Самый большой компонент должен хранить все N элементов, поэтому общее число компонентов равно $k = O\left(\log_f\left(\frac{N}{M}\right)\right)$, если наименьший компонент имеет размер $\theta(M)$.



Слияние меньшего компонента в больший в политике поуровневого слияния LSM-дерева. В примере показаны В+-древopodobные единичные компоненты на каждом уровне, но в современных реализациях LSM-дерева вместо В+-деревьев используются различные структуры данных или даже простые таблицы и файлы

LSM-деревья

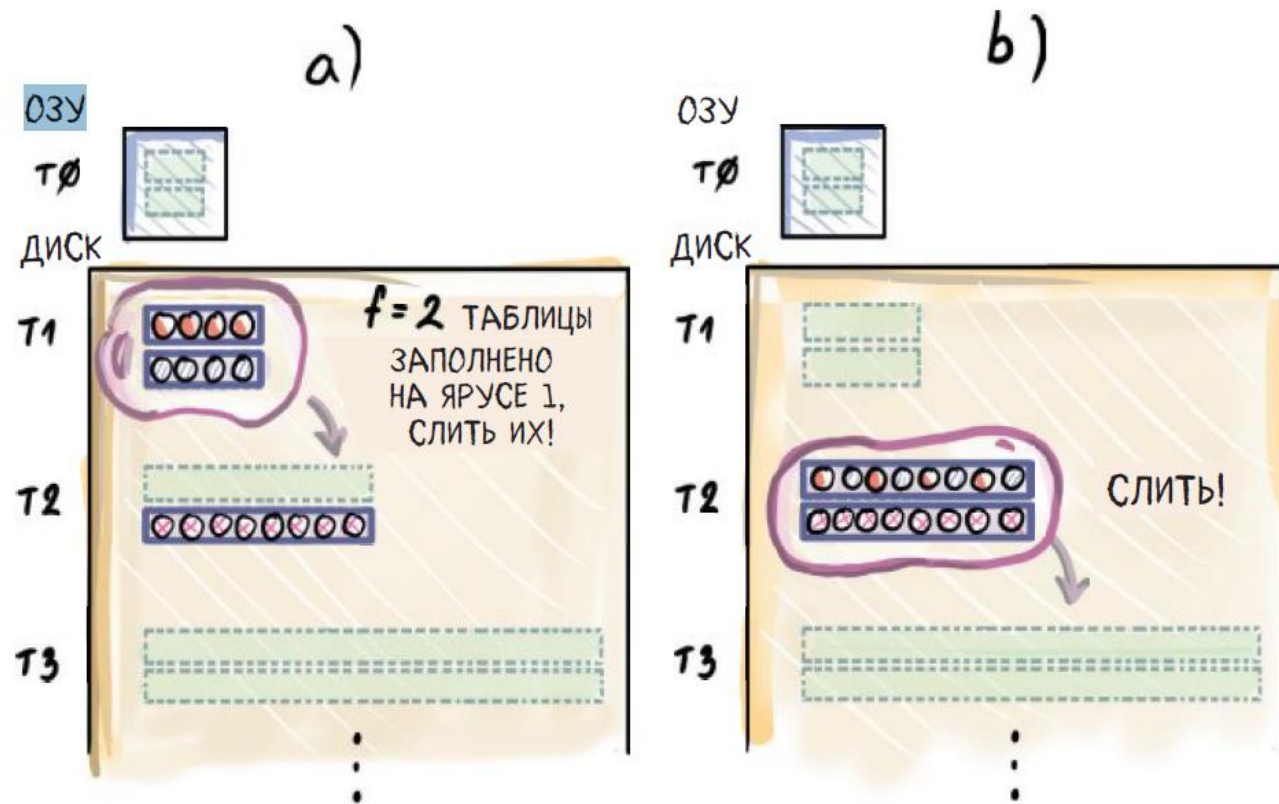
Обратите внимание, что при таком слиянии для спуска одного элемента на следующий уровень мы записываем этот фрагмент данных многократно:

- один раз, когда он сливается в более низкий уровень, а затем позже, когда другие элементы сливаются в его уровень.

Этот процесс повторяется для каждого уровня, и этот так называемый эффект *усиления операции записи*. Термин *усиление операции записи* используется для измерения объема данных, который записывается внутри структуры данных на единицу вставленного элемента.

Политика поярусного слияния — еще один популярный механизм уплотнения компонентов в современных реализациях LSM-дерева. В данной политике ярусы эквивалентны уровням, за исключением того, что каждый ярус содержит f компонентов одинакового размера.

Как только f компонентов на ярусе i заполнены, все они сливаются в один новый компонент на ярусе $i + 1$. Благодаря такому подходу для спуска на новый ярус каждый элемент записывается только один раз.



Политика поярусного слияния в LSM-дереве с $f = 2$.

Когда f компонентов на ярусе i заполняются, они сливаются в один компонент на ярусе $i + 1$.

Анализ стоимости LSM-дерева

Обратите внимание, что при обеих политиках слияния мы теряем часть первоначальной $O(1/B)$ производительности записи во время сжатия. Поскольку при политике поуровневого слияния каждый элемент записывается примерно $O(f)$ раз, чтобы спуститься на один уровень, общая стоимость спуска одного элемента на один уровень ниже равна $O(f/B)$, а общая стоимость спуска вниз LSM-дерева – это стоимость, накопленная на всех уровнях: $O\left(\left(\frac{f}{B}\right) \log_f \frac{N}{M}\right)$ операций ввода-вывода. При политике поярусного слияния нам нужно только $O(1/B)$ на каждый уровень, и в совокупности по всем уровням требуется $O(\log_f \frac{N}{M})$ операций ввода-вывода, что в f раз меньше, чем при политике поуровневого слияния.

Поскольку в политике поуровневого слияния число компонентов равно числу уровней, для поиска требуется $O(\log_f \frac{N}{M})$ операций ввода-вывода. В политике поярусного слияния приходится проверять в f раз больше компонентов, что увеличивает стоимость запроса. Однако наибольший прирост производительности запросов достигается за счет использования фильтров Блума, которые помогают перенаправлять запрос в нужную таблицу, тем самым в большинстве случаев доводя производительность запроса до $O(1)$. Эта оптимизация может применяться к обеим политикам слияния, и на этот раз она работает, поскольку общее число компонентов логарифмично по отношению к общему размеру набора данных, и, следовательно, поддержание фильтров Блума в основной памяти вполне управляемо. Диапазонным запросам повезло меньше, так как они не могут также эффективно использовать фильтры Блума, как точечные запросы.

Вариант использования: LSM-деревья в Cassandra

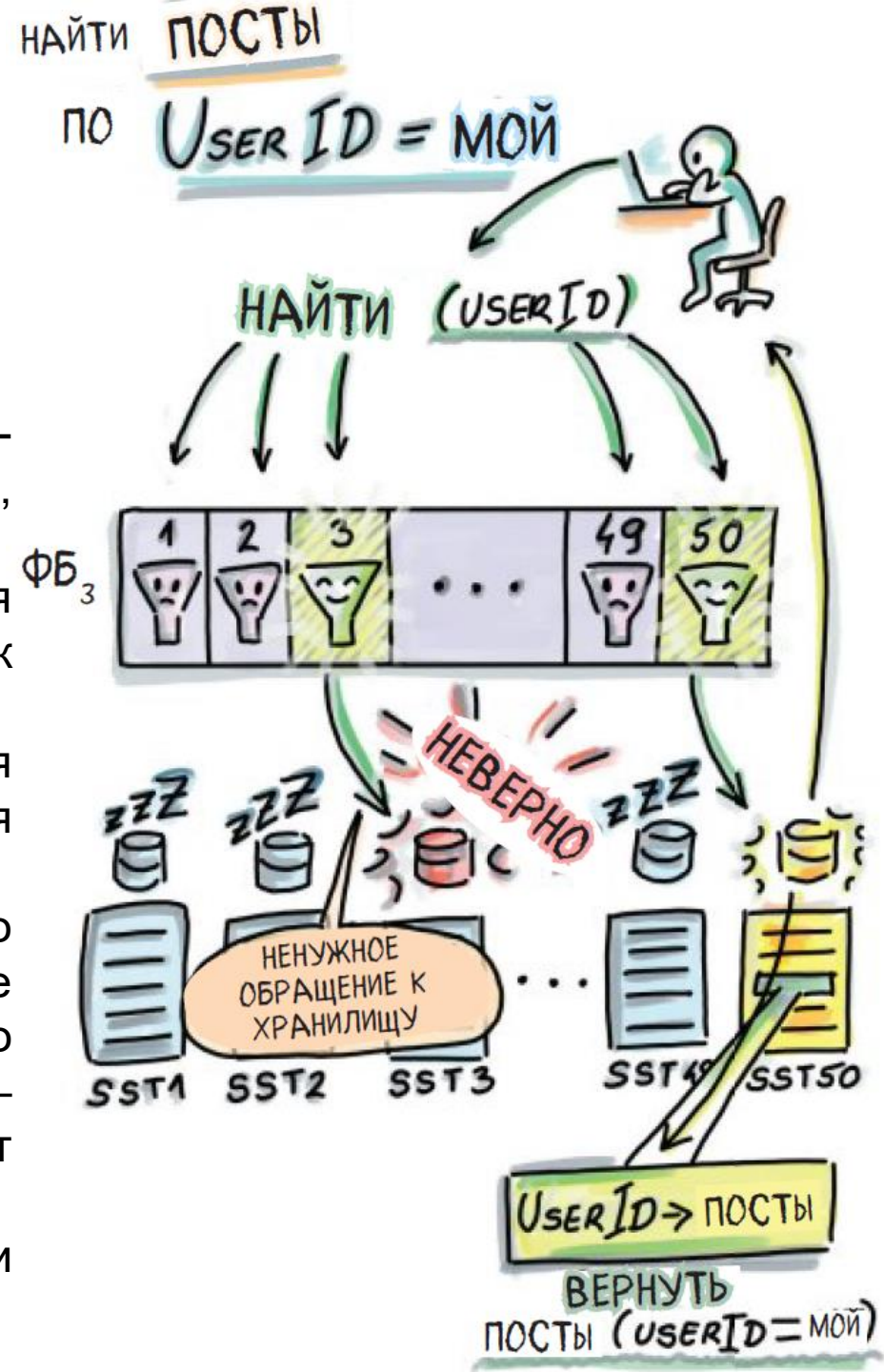
LSM-деревья были реализованы в ряде крупных движков баз данных, таких как Cassandra, LevelDB, RocksDB и т. д. В частности, LSM-дерево поярусного слияния в Cassandra использует фильтры Блума, чтобы избегать ненужных операций дискового поиска.

На рисунке показано LSM-дерево, компонентами которого являются таблицы. Наглядно видно, что при $f = 4$ таблицы SST 1-4 относятся к первому ярусу, SST 5-8 – ко второму ярусу и т. д.

Типичным вариантом использования LSM-дерева является приложение, которому требуется невероятно высокая производительность операций записи.

Примером такого приложения является продукт резервного копирования, который делает снимки данных через регулярные промежутки времени и хранит петабайты данных, но редко пересматривает их историю. Еще один вариант использования – сетевое приложение для мониторинга трафика, которое отслеживает сотни миллионов запросов в час.

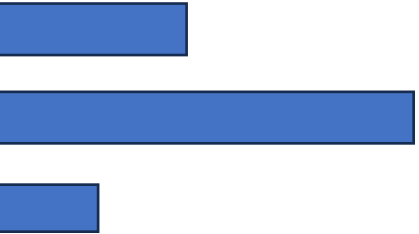
Запросы хранятся в базе данных, но случаи явного поиска того или иного запроса бывают редко.



COLA



Cache-oblivious lookahead array - кэш-неосведомлённый массив с заглядыванием вперёд
Хотим поиск в упорядоченном словаре; COLA близка к реализации LSM (конкурент)



Сегменты – упорядоченные по возрастанию ключей поток. Имеем иммутабельные сегменты и не иммутабельный набор сегментов.

Размер сегмента $size = 2^i$, притом все i – разные

Например COLA – 11 = 1 + 2 + 8.

Хочу например 11 + 1 = $1011_2 + 0001_2 = 1100_2 \rightarrow$ должны слить 1 + 1 и 2 + 2

Работаем тупо как с двоичным сложением. Вставка стоит $\frac{1}{B} \log_2 N$

Вставка в COLA:

Fractional Cascading

$A_0, A_1, \dots, A_{l-1}$



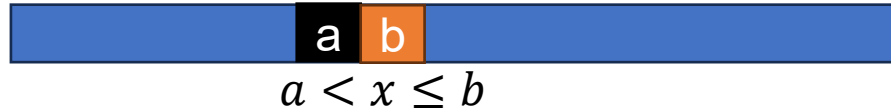
$|A_i| = n$

Сольем например все массивы в 1

$$B = \prod_i A_i$$

Хочу вместо $O(l \cdot \log n) \rightarrow O(l + \log n)$

$$\left\{ \begin{array}{l} A'_0 = A_0 \\ A'_1 = A_1 \cup \frac{1}{2} A'_0 \\ A'_2 = A_2 \cup \frac{1}{2} A'_1 \\ \vdots \\ A'_{l-1} = A_{l-1} \cup \frac{1}{2} A'_{l-2} \end{array} \right.$$

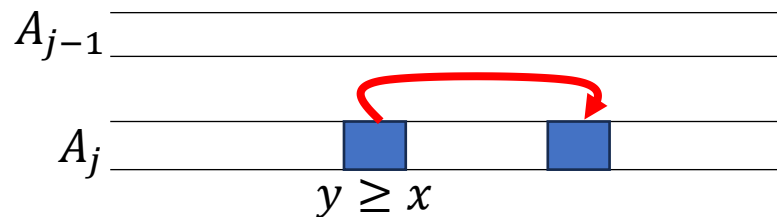


Если сделать быстро сложно понять как дальше\\

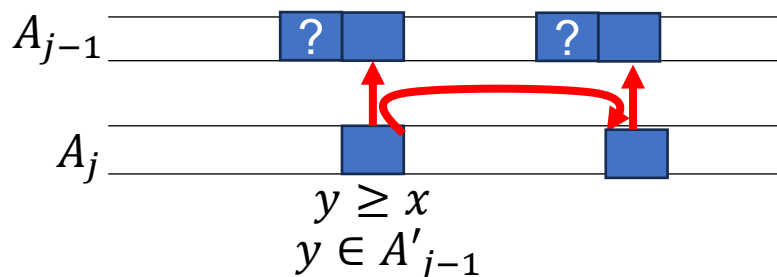
$\frac{1}{2}$ - значит берем строго каждый второй элемент

Сколько в итоге стало элементов - $A_0 + \frac{1}{2} A_0 + \dots + \frac{1}{2^l} A_0 < 2A_0$

COLA



Если $y \in A_j$ - ура, если $y \notin A_j$ - можем предпросчитывать стрелку на следующий $y \in A_j$



Если $y \in A'_{j-1}$ - то мы можем посмотреть левого соседа элемента из A_{j-1} - так как мы прореживаем каждого 2, Если $y \in A'_j$ - можем предпросчитывать стрелку на следующий $y \in A_{j-1}$ - и смотрим снова левого соседа

Проблема часто вставки, значит часто меняется A_0 , а значит постоянно перестраивать Fractional Cascading. Как победить? Представлять наоборот..

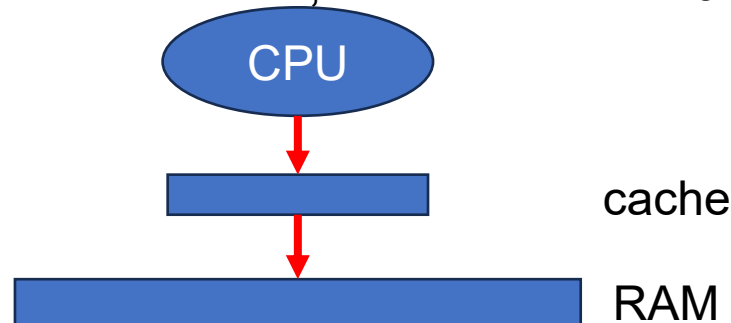


$|A'_0| = 2^{l-1}$; $|A'_1| = 2^{l-2} + 2^{l-2} = 2^{l-1}$ - пока нет экспоненциального убывания размера, то есть общий размер как будто очень большой и асимптотика плохая

$$\left\{ \begin{array}{l} A'_0 = A_0 \\ A'_1 = A_1 \cup \frac{1}{8}A'_0 \\ A'_2 = A_2 \cup \frac{1}{8}A'_1 \\ \vdots \\ A'_{l-1} = A_{l-1} \cup \frac{1}{8}A'_{l-2} \end{array} \right. \quad \begin{array}{l} |A'_0| = 2^{l-1}; \\ |A'_1| = 2^{l-2} + 2^{l-4} < 2^{l-1} \\ |A'_2| = 2^{l-3} + 2^{l-5} < 2^{l-2} \\ |A'_3| = 2^{l-4} + 2^{l-6} < 2^{l-3} \end{array}$$

Есть асимптотика –
есть индукция

Отличие от внешней памяти модели – не можем управлять кэшем, $B = \text{cache line} = 64\text{-}128 \text{ bytes}$



PGM - индекс

Статья [по ссылке](#)Библиотека [по ссылке](#)

key: int64

value: $\forall$

index

Будем считать например упорядоченным. Бин поиск уже проблематичен (внешняя память, где какая строка заканчивается). Но как-то ищем $\log_2 N$. Если сделать B+-tree, то $\log_B N$. Хотим search [], lower_bound, ...

index – компактен, помогает искать быстрее. Память хотим в сделать меньше чем B+-tree, но скорость поиска такую же.

Piecewise Geometric Model Index —кусочно-линейная геометрическая модель

Нужна 1 функция: rank: $K \rightarrow \mathbb{N}$, где K множество ключей

Датасет состоит из ключей $D = \{a_0, \dots, a_{N-1}\}$

$$\text{Rank}(x) = \#\{i \mid a_i < x\}$$

Пример: $\text{Rank}(a_i) = i$

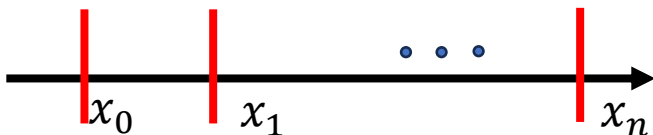
Идея 1: $\text{Rank}(x) = \alpha x + \beta$ – подойдет как отрезок для ключей арифметической прогрессии.

Здесь часть индекса α и β .

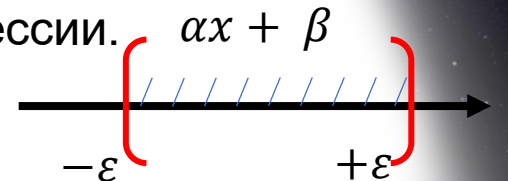
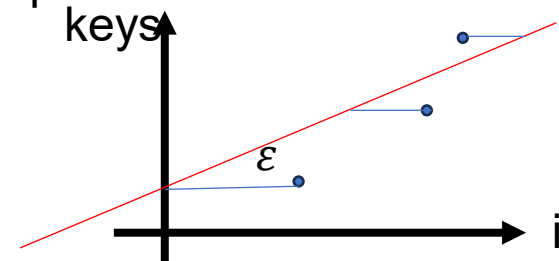
Идея 2: $\forall x \in [x^-, x^+] \mid |\text{Rank}(x) - |\alpha x + \beta|| \leq \varepsilon$

Как посчитать rank. Мы знаем ε окрестность нашего ответа, далее перебором в окне – либо чтение блока, если внешняя память, либо бинпоиск если во внутренней.

Идея 3: Разбить на сегменты $\forall x_i \in [x_i, x_{i+1}] \mid |\text{Rank}(x) - |\alpha_i x + \beta_i|| \leq \varepsilon$



Самый большой синий отрезок - ε



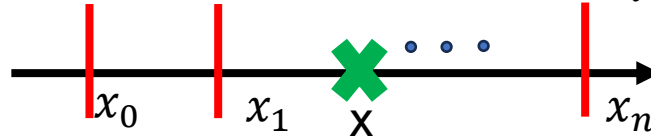
PGM - индекс

Задача:

Берем датасет и фиксируем ε . Хотим разбить на сегменты так и найти α_i и β_i такие, что

$$\forall x_i \in [x_i, x_{i+1}] |Rank(x) - |\alpha_i x + \beta_i|| \leq \varepsilon$$

index состоит из x_i , α_i и β_i



Задача оптимизация памяти: Разбить на отрезки оптимально.

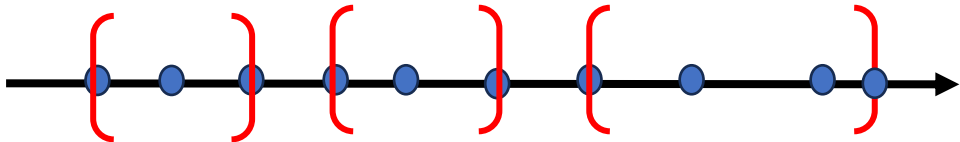
Как искать в индексе x ? Поиск сегмента – задача вычисления ранга (рекурсия).

Верхняя оценка на отрезки: $\frac{N}{2\varepsilon} + O(1)$. Придумаем такую (тупую) конфигурацию

Берем $rank = \text{const}$ и делим на отрезки длиной 2ε . По факту index выразится в В-дерево с количеством в блоке значений $\sim \varepsilon$. Значит индекс точно не проигрывает В дереву.

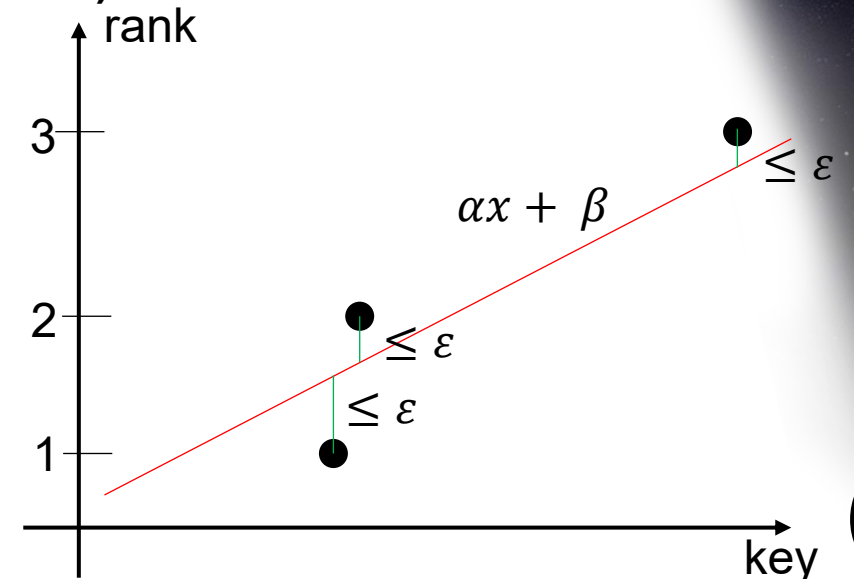
Разбить на отрезки оптимально. (делали в 1ом семестре, напомним)

1 часть: минимальное количество отрезков с ошибкой не больше 2ε



Решение: берем отрезок оптимальный из первых точек самый длинный, далее по индукции легко показать, что 2 отрезок максимально длинный после первого и т.д.

Сколько стоит найти оптимальное решение – $O(N)$. Как проверить, что отрезок хороший?



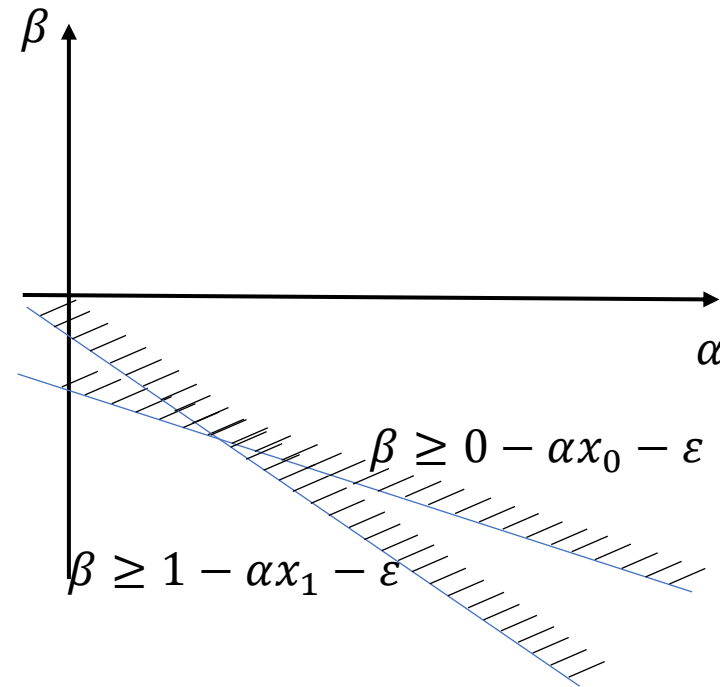
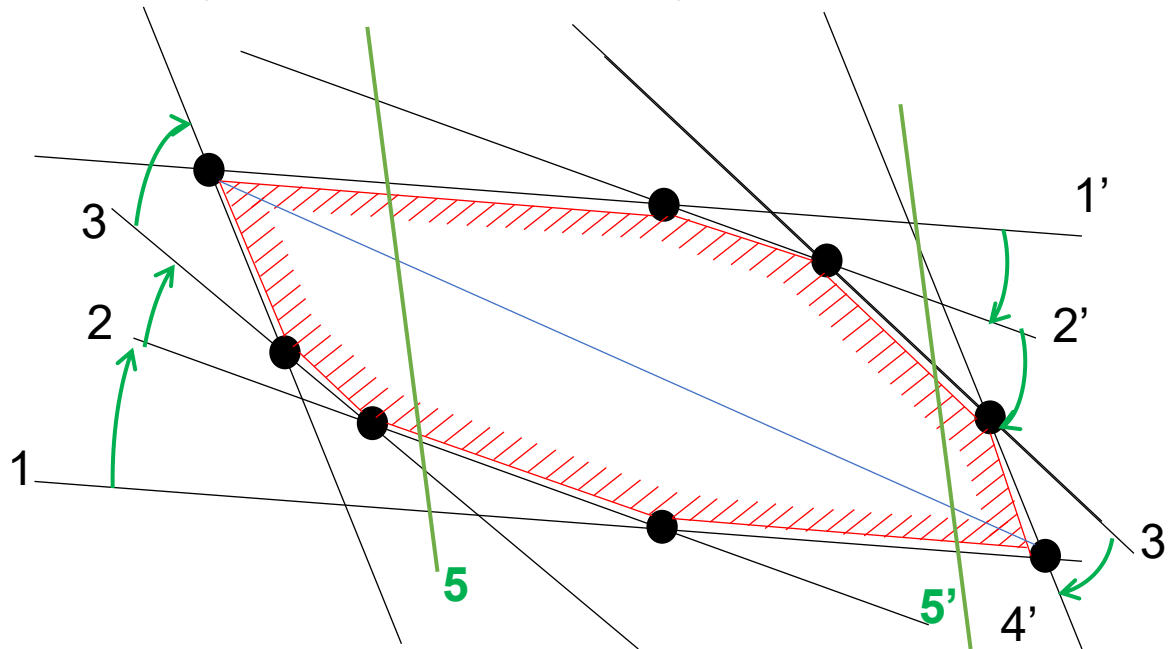
PGM - индекс

Как инкрементально добавлять точки в датасет.

Ограничения α, β : $\forall i \quad |\alpha_i x + \beta_i - i| \leq \varepsilon \Leftrightarrow \begin{cases} \alpha_i x + \beta_i - i \geq -\varepsilon \\ \alpha_i x + \beta_i - i \leq \varepsilon \end{cases}$

Эти условия задают многоугольник, внутри которого лежат точки. Добавляются новые неравенства, которые геометрически - пересекающие полуплоскости. Условие после этого точки должны остаться внутри фигуры

$$4 \quad \begin{cases} \alpha_i x + \beta_i - i \geq -\varepsilon \\ \alpha_i x + \beta_i - i \leq \varepsilon \end{cases} \Leftrightarrow \begin{cases} \beta_i \geq i - \alpha_i x - \varepsilon \\ \beta_i \leq i - \alpha_i x + \varepsilon \end{cases}$$



Пример 4 пары параллельных прямых – внутри нужная фигура красным цветом.

Задача:

Перед нами стоит задача вычислительной геометрии. Добавить условие к многоугольнику плоскость.

Доп. Условие которое дает наша задача: 5 прямая дает по 1 точке пересечения в верхнем и нижнем полупериметрах.

Нужен deque для решения задачи пересечения для нашей задачи. Остановка, когда фигура станет пустой.